Tobigs 25 & 26th

MLOps & Database

7주차 2교시 | 25기 김지은

Contents

SECTION 01

Why MLOps

ML System과 ML Code · Hidden Technical Debt · DevOps vs MLOps · Maturity Level

SECTION 02

실험 관리와 재현성

Versioning · Lineage · Experiment Tracking · Model Registry

SECTION 03

AI 서비스를 위한 Database

RDB / NoSQL · Vector DB · ANN · RAG · Feature Store

SECTION 04

모델의 서비스화

Container · Batch / Online Serving · 배포 전략

SECTION 05

Monitoring

Data Drift · Concept Drift · Label Delay

SECTION 06

자동화와 Continuous Training

Orchestration · CT · Rollback

SECTION 07

Quiz / Q&A

연구 환경 vs 실제 서비스

연구 환경의 좋은 성능이 서비스 환경의 좋은 성능을 보장하지 않는다

구분	연구 환경 (Research)	실제 서비스 (Production)
요구사항	벤치마크 데이터셋에서의 최고 성능	이해관계자(사용자·사업·운영)마다 다른 요구사항
계산 우선순위	빠른 학습, 높은 처리량(Throughput)	빠른 추론, 낮은 지연시간(Latency)
데이터	고정된(static) 데이터셋	계속 변하는 데이터
공정성	주요 고려 대상이 아닌 경우가 많음	반드시 고려
해석 가능성	주요 고려 대상이 아닌 경우가 많음	반드시 고려

핵심 질문: **테스트셋에서 정확도 97%인 모델은 서비스에서도, 앞으로도 계속 97%인가?**

Chip Huyen, *Designing Machine Learning Systems* (O'Reilly, 2022), Ch.1 "Machine Learning in Research Versus in Production"의 비교 항목을 요약

핵심 원리

ML System = Code + Data + Model + Environment, 네 요소 모두 변화함

MLOps = 변화를 **기록**하고, **통제**하고, **감지**하고, **대응**하는 기술

변하는 것	문제	다루는 개념	대표 도구 (예시)	파트
코드·하이퍼파라미터	어떤 설정이 좋았는지 추적 불가	Experiment Tracking, Versioning	Git, MLflow	2부
데이터	같은 코드로도 결과가 달라짐	Data Versioning, Lineage	DVC(Data Version Control), 메타데이터 스토어	2부
모델	현재 서비스 중인 모델 버전을 알 수 없음	Model Registry	MLflow Registry	2부
피처 계산 방식	학습 시점과 서빙 시점의 계산 방식이 다름	Feature Store, Training-Serving Skew	Feast	3부
참조 지식	모델에 최신 정보가 반영되지 않음	Vector DB, RAG	pgvector, Milvus, Pinecone	3부
실행 환경	환경이 바뀌면 실행·결과가 달라짐	Image / Container	Docker	4부
트래픽	요청이 몰리면 응답 지연·장애	Serving, Scaling	FastAPI, BentoML, Kubernetes	4부
외부 환경 (데이터 분포)	에러 없이 예측 성능이 떨어짐	Data / Concept Drift, Monitoring	Evidently	5부
위 요소 전체	수작업으로는 반복 수행이 어려움	Orchestration, CT, Rollback	Airflow	6부

SECTION 01

Why MLOps

모델을 좋은 성능으로 학습시켰는데 왜 끝이 아닌가?

ML System과 ML Code는 같지 않다

실제 ML 시스템에서 ML 코드가 차지하는 비중은 작음

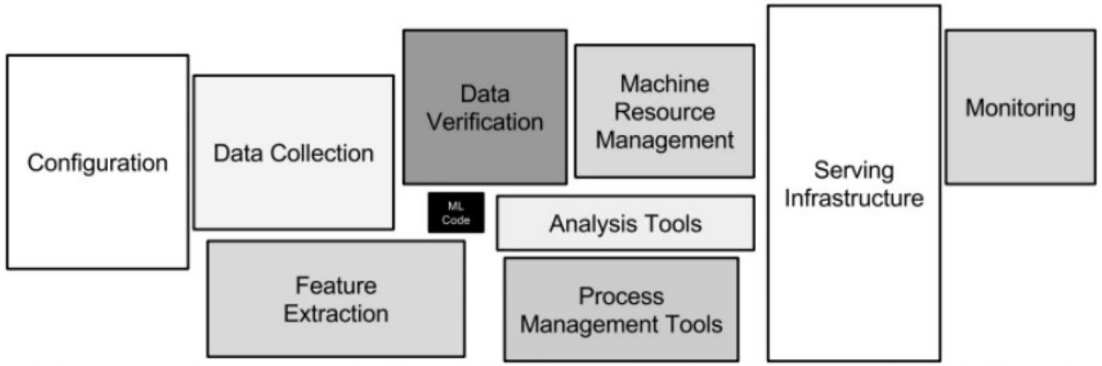


Figure 1: Only a small fraction of real-world ML systems is composed of the ML code, as shown by the small black box in the middle. The required surrounding infrastructure is vast and complex.

> *"The real challenge isn't building an ML model, the challenge is building an integrated ML system and to continuously operate it in production."*
>
> 핵심 과제는 ML 모델 구축이 아니라, 통합된 ML 시스템을 구축하고 프로덕션에서 지속적으로 운영하는 것이다.
>
> — Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning"

- 중앙의 검은 박스(ML Code)를 둘러싼 구성 요소: Configuration, Data Collection·Verification, Feature Extraction, Resource Management, Serving Infrastructure, Monitoring …

Sculley et al., "Hidden Technical Debt in Machine Learning Systems", NeurIPS 2015, Figure 1 / Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning"

MLOps란?

MLOps는 모델 하나가 아니라, 모델을 계속 만들어 운영하는 시스템을 다루는 방법론

> *"MLOps is an ML engineering culture and practice that aims at unifying ML system development (Dev) and ML system operation (Ops)."*
>
> ML 시스템 개발(Dev)과 운영(Ops)의 통합을 목표로 하는 ML 엔지니어링 문화이자 practice
>
> — Google Cloud MLOps 문서

> *"MLOps (Machine Learning Operations) is a paradigm, including aspects like best practices, sets of concepts, as well as a development culture when it comes to the end-to-end conceptualization, implementation, monitoring, deployment, and scalability of machine learning products."*
>
> ML 제품의 end-to-end 설계·구현·모니터링·배포·확장(scalability)에 걸친 best practice, 개념, 개발 문화를 포함하는 패러다임
>
> — Kreuzberger et al., IEEE Access 2023

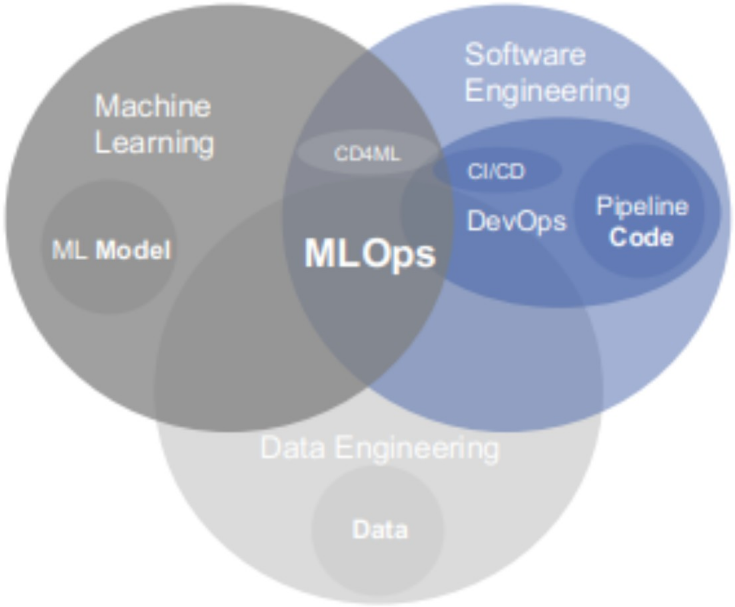


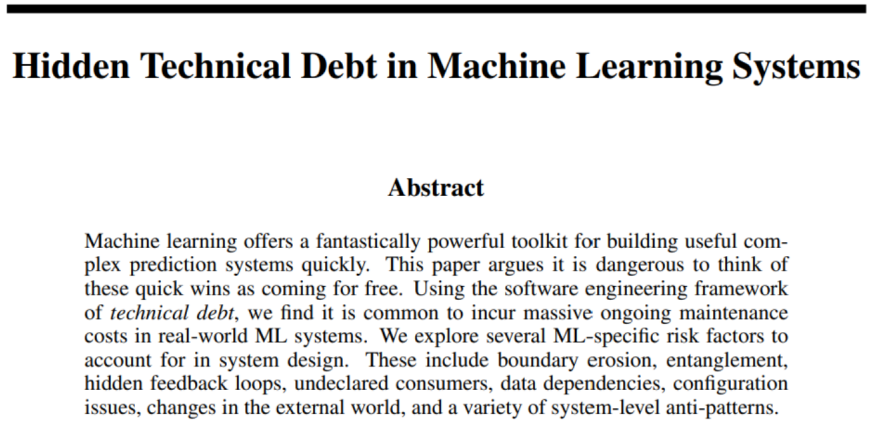
Figure 5. Intersection of disciplines of the MLOps paradigm

Kreuzberger et al. Fig.5 — MLOps는 ML·소프트웨어 공학(DevOps)·데이터 엔지니어링이 겹치는 영역

Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning" / Kreuzberger, Kühl, Hirschl, "Machine Learning Operations (MLOps): Overview, Definition, and Architecture", IEEE Access 2023

Hidden Technical Debt

ML 시스템은 일반 소프트웨어의 부채에 더해 ML 고유의 기술 부채를 가진다



Hidden Technical Debt in Machine Learning Systems

Abstract

Machine learning offers a fantastically powerful toolkit for building useful complex prediction systems quickly. This paper argues it is dangerous to think of these quick wins as coming for free. Using the software engineering framework of *technical debt*, we find it is common to incur massive ongoing maintenance costs in real-world ML systems. We explore several ML-specific risk factors to account for in system design. These include boundary erosion, entanglement, hidden feedback loops, undeclared consumers, data dependencies, configuration issues, changes in the external world, and a variety of system-level anti-patterns.

Sculley et al. (2015) 제목과 Abstract

- ML 시스템에는 일반 소프트웨어의 부채에 더해 **ML 고유의 부채**가 존재
- Abstract 요지: 빠른 성과(quick wins)에 비용이 따르지 않는다고 보는 것은 위험함
- 기술 부채(Technical Debt): 빠른 개발을 위해 택한 설계상의 타협이 이후 유지보수 비용으로 누적되는 것

> *"Machine learning systems mix signals together, entangling them and making isolation of improvements impossible. … If we change the input distribution of values in x_1, the importance, weights, or use of the remaining $n - 1$ features may all change. … We refer to this here as the CACE principle: Changing Anything Changes Everything."*
>
> ML 시스템에서는 입력 피처들이 서로 얽혀 있어(entanglement) 한 부분만 따로 개선하기 어렵다. x_1의 분포를 바꾸면 나머지 n−1개 피처의 중요도·가중치·사용 방식이 모두 바뀔 수 있다 → **CACE(Changing Anything Changes Everything)**
>
> — Sculley et al., 2015, § 2.1 Entanglement

Sculley et al., "Hidden Technical Debt in Machine Learning Systems", NeurIPS 2015

DevOps vs MLOps

ML 시스템도 소프트웨어지만, 동작이 코드가 아닌 데이터에 의해 결정됨

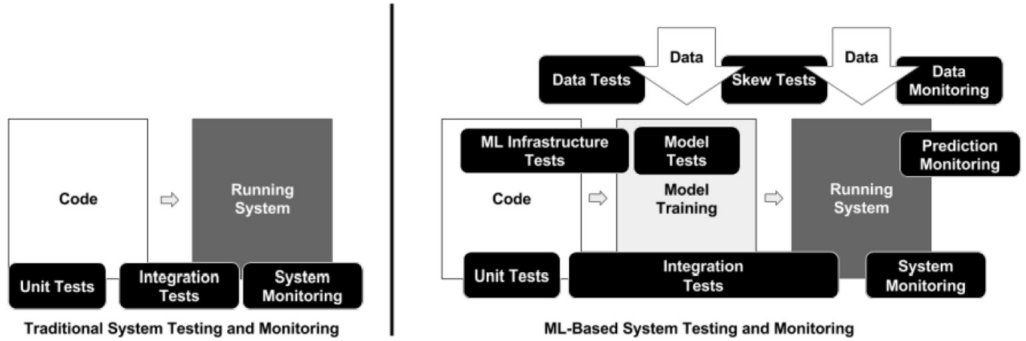


Figure 1. **ML Systems Require Extensive Testing and Monitoring.** The key consideration is that unlike a manually coded system (left), ML-based system behavior is not easily specified in advance. This behavior depends on dynamic qualities of the data, and on various model configuration choices.

Breck et al. (2017) Figure 1 — 수동 코딩 시스템(좌) vs ML 기반 시스템(우)의 테스트·모니터링 범위

> ML 모델은 코드 문제뿐 아니라 계속 변하는 data profile 때문에도 성능이 떨어진다.
>
> — Google Cloud MLOps 문서, "DevOps versus MLOps"

Data Drift: 서비스 중 들어오는 입력 데이터의 분포가 학습 때와 달라지는 현상. 코드에 오류가 없어도 성능이 떨어지는 대표적인 원인

→ 5부 Monitoring에서 자세히

구분	일반 소프트웨어 (DevOps)	ML 시스템 (MLOps)
팀	소프트웨어 엔지니어 중심	데이터 과학자·연구자 포함
개발	정해진 로직을 구현	**실험적**: 피처·알고리즘·하이퍼파라미터를 계속 시도
테스트	유닛·통합 테스트	+ **데이터 검증**, 모델 품질 평가, 모델 검증
배포	서비스 하나를 배포	**재학습·재배포 파이프라인**을 배포
성능 저하 원인	코드 결함	코드 결함 + **변하는 데이터 분포** (model decay)

Google Cloud MLOps 문서, "DevOps versus MLOps" / Breck et al., "The ML Test Score", IEEE BigData 2017, Figure 1

대표적인 ML 기술 부채 4가지

성능 저하의 주된 원인은 모델 코드 밖(데이터·파이프라인·외부 환경)에 있다

부채	발생하는 문제	다루는 파트
Data Dependency	코드와 달리 데이터는 리뷰·버전 관리 대상에서 빠지기 쉬움. 서비스 중 입력 데이터가 바뀌면 모델의 예측 성능이 떨어짐	2부 Versioning, 5부 Monitoring
Pipeline Jungle	수집·전처리·피처 스크립트가 임시방편으로 얽혀 전체 구조를 파악·관리하기 어려움	3부 Feature Store, 6부 Orchestration
Training-Serving Skew	학습 시점과 서빙 시점의 데이터 처리 방식이 달라 성능 차이 발생	3부 Feature Store
External World의 변화	데이터·모델·코드가 계속 변하므로 한 번 학습한 모델을 그대로 유지할 수 없음	5부 Drift, 6부 CT

> Training-Serving Skew는 Sculley 논문이 아니라 Google *Rules of ML*의 용어. 원인: ① 학습·서빙 파이프라인의 데이터 처리 차이 ② 학습~서빙 사이 데이터 변화 ③ 모델-알고리즘 간 피드백 루프
>
> — Zinkevich, Rules of Machine Learning (Google)

Sculley et al., NeurIPS 2015 / Zinkevich, “Rules of Machine Learning: Best Practices for ML Engineering”, Google

CI / CD / CT

ML에서는 CI·CD의 대상이 데이터·모델까지 넓어지고, 자동 재학습(CT)이 추가된다

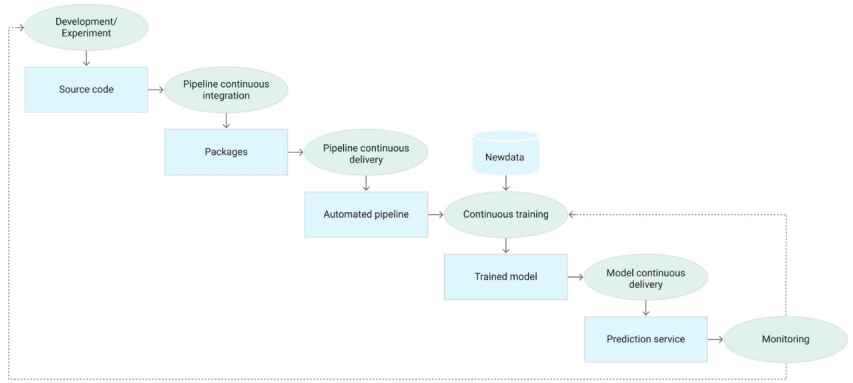


Figure 5 — 코드 → 파이프라인 CI/CD → 자동 학습(CT) → 모델 배포 → 모니터링 순서

- Figure 5의 흐름: Pipeline CI → Pipeline CD → **Continuous Training** → Model CD → Monitoring

DevOps = CI + CD → MLOps = CI + CD + **CT**

CI (Continuous Integration)

"CI is no longer only about testing and validating code and components, but also testing and validating data, data schemas, and models."

코드뿐 아니라 **데이터·스키마·모델**까지 테스트·검증

CD (Continuous Delivery)

"CD is no longer about a single software package or a service, but a system (an ML training pipeline) that should automatically deploy another service (model prediction service)."

예측 서비스를 자동 배포하는 **학습 파이프라인**을 배포

CT (Continuous Training)

"CT is a new property, unique to ML systems, that's concerned with automatically retraining and serving the models."

ML 시스템에만 있는 속성. **자동 재학습·서빙**

Google Cloud, "MLOps: Continuous delivery and automation pipelines in machine learning", "DevOps versus MLOps", Figure 5 (CC BY 4.0)

MLOps 성숙도 Level 0 — 수동 프로세스

| 모든 단계가 수동이라, 배포 이후 모델이 관리되지 않는다

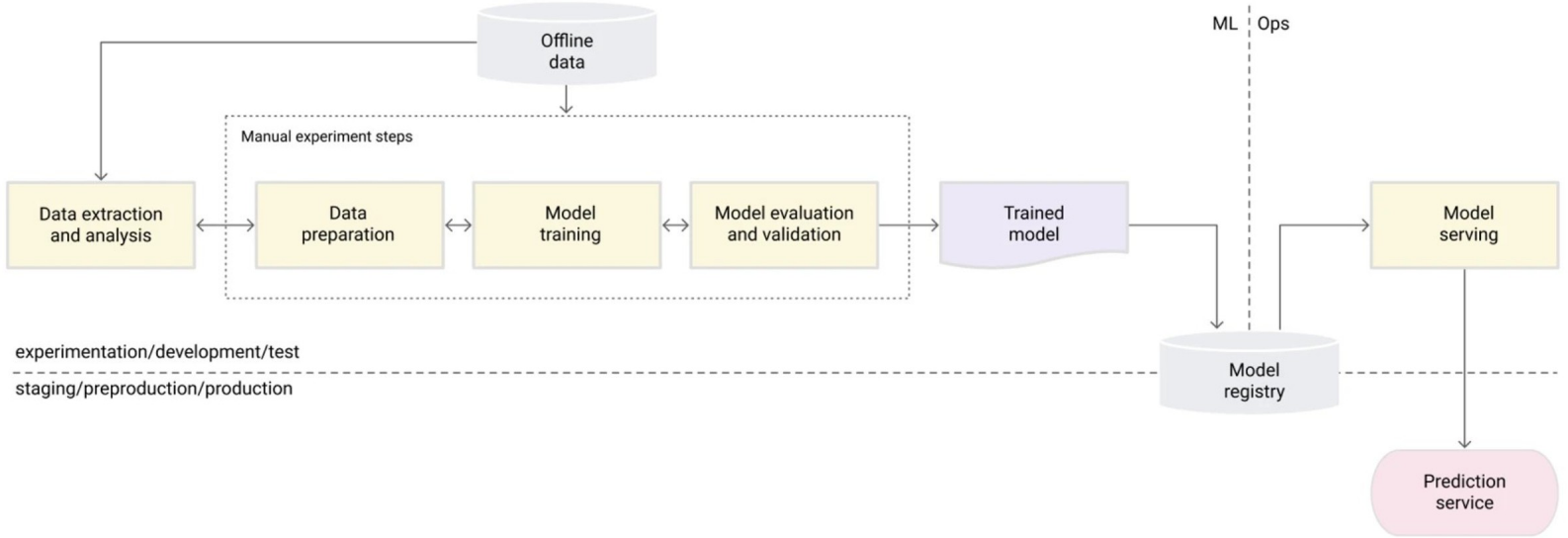


Figure 2 — 실험부터 서빙까지 모두 수동, ML|Ops 점선을 넘어 모델 파일만 전달됨

- 모든 단계가 **수동**: 노트북에서 실험 → 선택한 모델 가중치를 파일 형태로 전달
- 데이터 과학자(모델) ↔ 엔지니어(서빙) **단절** (그림의 ML | Ops 점선) → Training–Serving Skew 발생 지점
- 배포 빈도가 낮음 → CI/CD 없음, **성능 모니터링 없음**

Google Cloud, “MLOps: Continuous delivery and automation pipelines in machine learning”, Figure 2 (CC BY 4.0)

Level 1 — ML 파이프라인 자동화 (CT 달성)

배포 대상이 모델에서 파이프라인으로 바뀜

- 목표: 파이프라인을 자동화해 **Continuous Training** 달성
- 새로 추가되는 구성 요소
  - **Data & Model Validation**: 재학습 전 데이터 이상 확인, 재학습 후 기존 모델보다 나은지 확인
  - **Feature Store** (선택): 학습·서빙이 같은 피처 소스 사용
  - **ML Metadata Store**: 모든 실행 기록 → 재현성, lineage, 롤백
  - **Pipeline Trigger**: 재학습 시점 결정
- 실험·운영 환경이 같은 파이프라인 코드 사용 (experimental-operational symmetry)

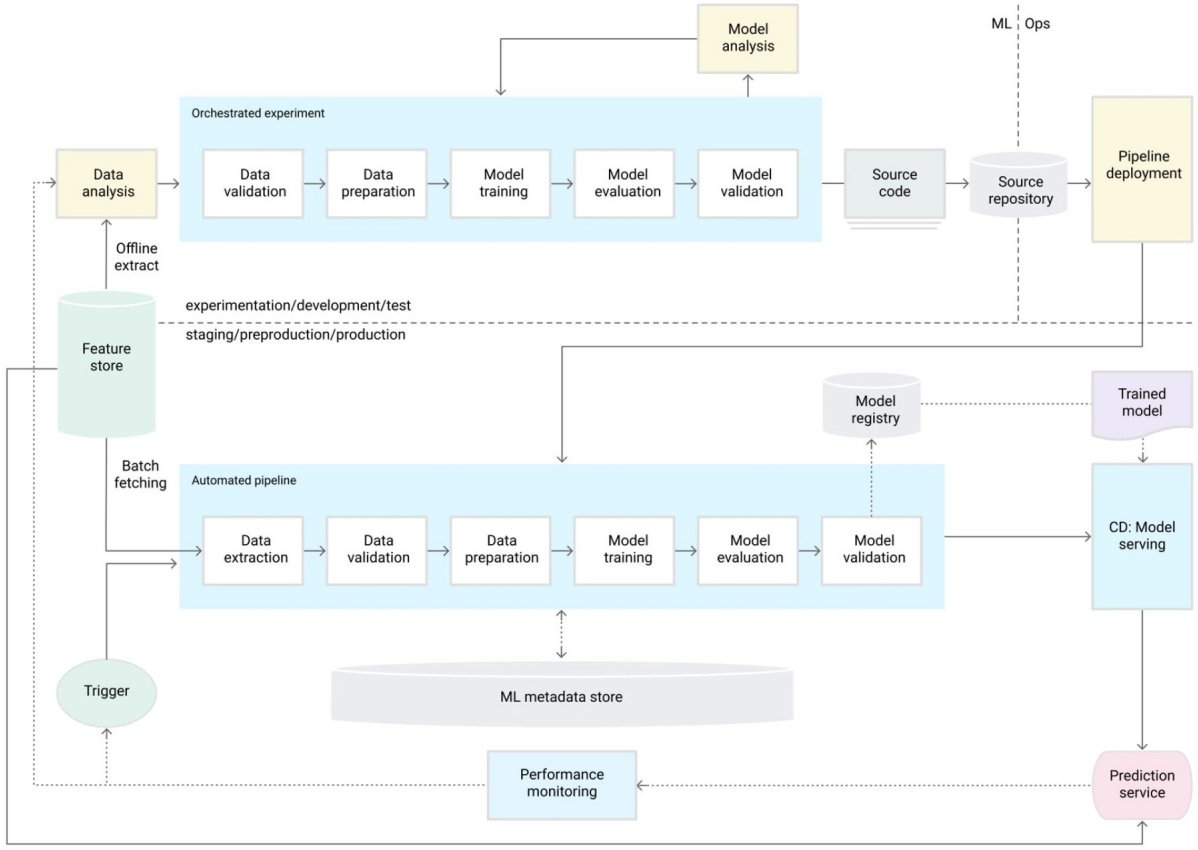


Figure 3 — 실험(위)과 운영(아래)이 같은 파이프라인을 쓰고, Trigger가 운영 파이프라인을 자동 실행

Google Cloud, “MLOps: Continuous delivery and automation pipelines in machine learning”, Figure 3 (CC BY 4.0)

Level 2 — CI/CD 파이프라인 자동화

코드 변경까지 자동으로 빌드·테스트·배포

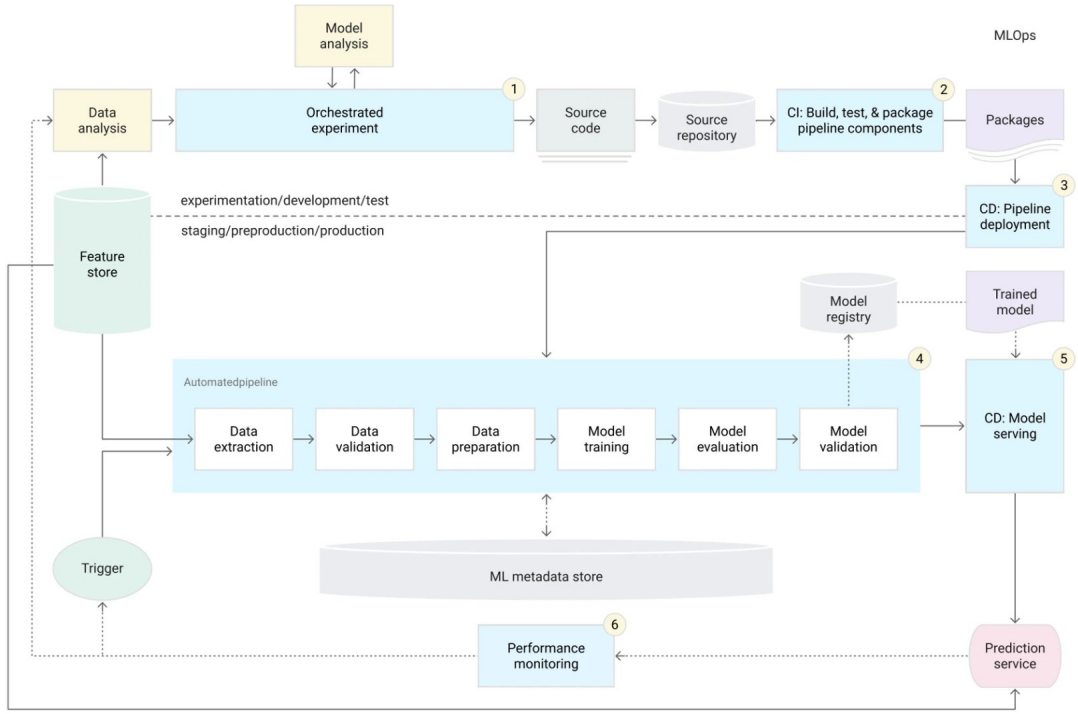


Figure 4 — 코드가 바뀌면 파이프라인 자체를 빌드·테스트·배포 (①~⑥ 단계)

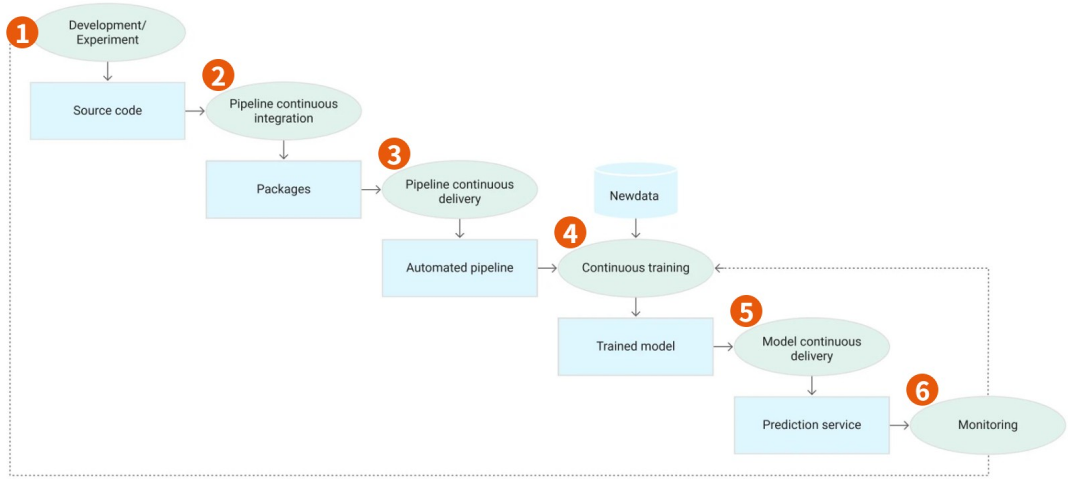


Figure 5 — 숫자는 Figure 4의 ①~⑥ 단계 (⑥ Monitoring → 다시 ①·④로)

- Level 1: 새 **데이터**로 재학습 자동화 / Level 2: 새 **코드(피처·구조·하이퍼파라미터)**도 자동 빌드·테스트·배포
- **한 번에 Level 2를 갖출 필요 없음.** 팀 상황에 맞춰 점진적으로 도입

Google Cloud, “MLOps: Continuous delivery and automation pipelines in machine learning”, Figure 4 · 5 (CC BY 4.0)

SECTION 02

실험 관리와 재현성

과거의 실험 결과를 동일하게 다시 만들 수 있는가?

재현성 (Reproducibility)

결과를 결정하는 4가지를 전부 고정·기록한다

- 재현성 = 같은 입력으로 **같은 결과를 다시 만들 수 있는 능력**

결과를 결정하는 요소	기록 방법
코드	Git commit hash
데이터	데이터 버전 (스냅샷·해시)
환경	라이브러리 버전, 컨테이너 이미지
무작위성	Random seed, 하이퍼파라미터

> 하나라도 누락되면 과거 결과 재현 불가. seed를 고정해도 일부 GPU 연산은 비결정적일 수 있음 → “비트 단위 재현”과 “통계적 재현”은 같지 않음

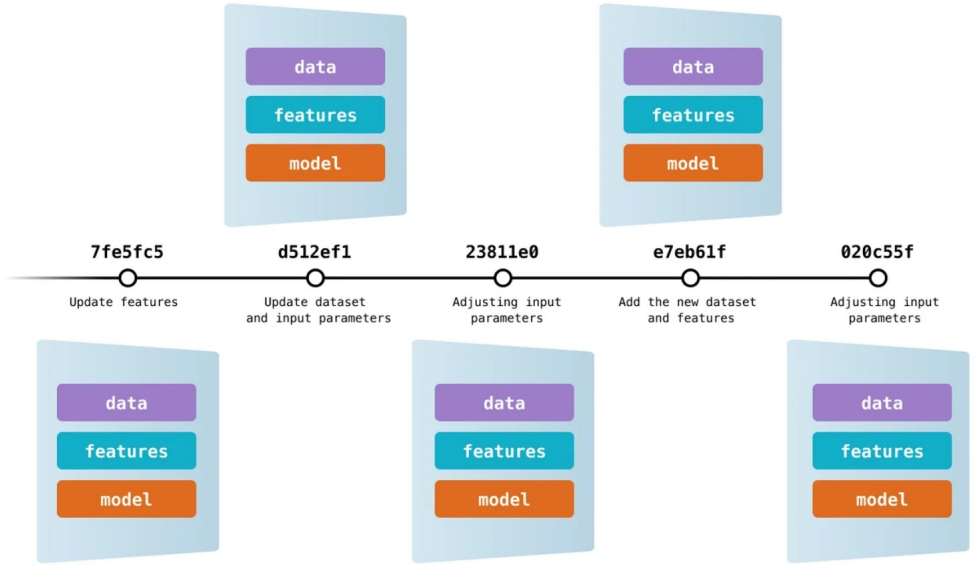
Google Cloud MLOps 문서 (Metadata management) / Kreuzberger et al., 2023 (Principles: Reproducibility, Versioning)

Data · Code · Model Versioning

버전 관리 대상을 코드에서 데이터·모델로 확장

- **Code Versioning**: Git
- **Data Versioning**: 대용량 데이터는 Git으로 관리하기 어려움 → 데이터는 외부 저장소에, Git에는 **메타파일(해시)**만 (예: DVC)
- **Model Versioning**: 모델 파일 + 그 모델을 만든 코드·데이터·파라미터 버전을 **묶어서** 기록

> 추적할 데이터셋·아티팩트를 기술한 메타파일을 만들고, 큰 파일 대신 이 메타데이터를 Git에 넣는다.
>
> — DVC Docs, “Versioning Data and Models”



Git 커밋마다 data · features · model 버전이 함께 기록됨

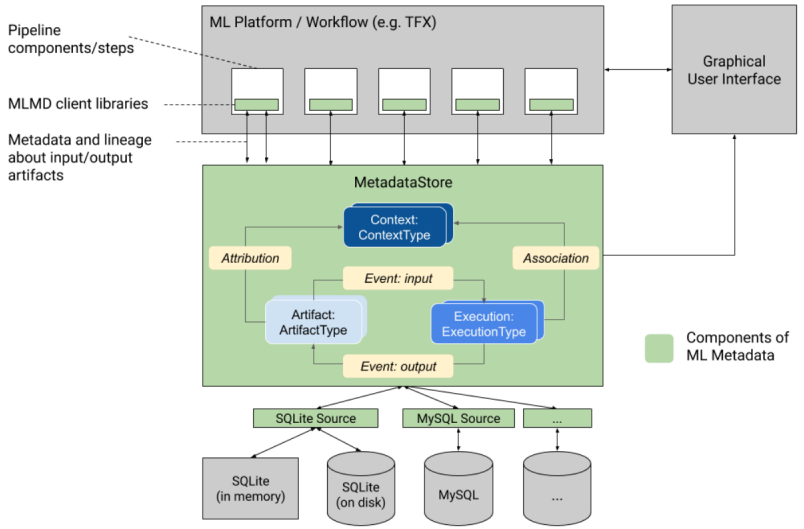
DVC Docs, “Versioning Data and Models” (doc.dvc.org), project-versions 그림

Data Lineage와 ML Metadata Store

모델이 어떤 데이터·변환·코드로 만들어졌는지 추적

- **Data Lineage**: raw 데이터에서 모델·배포까지 각 단계의 **입력 · 코드 · 설정 · 출력** 버전을 연결한 기록
- 문제 발생 시 이 기록을 **역추적**해 원인 단계를 파악

ML Metadata Store: 파이프라인 실행 정보를 저장하는 저장소. Google 문서 기준 기록 항목 — 파이프라인·컴포넌트 버전, 실행 시각과 소요 시간, 실행자, 파라미터, 단계별 아티팩트 포인터, 이전 모델 포인터, 평가 지표



ML Metadata(MLMD) 구조: Artifact · Execution · Context와 그 관계 저장

단계	입력	코드 (commit)	설정	출력
정제	raw_data v3	clean.py @a1b2c	—	train.parquet v7
학습	train.parquet v7	train.py @d4e5f	lr=0.01, seed=42	model v12
배포	model v12	serve.py @g7h8i	image tag 1.4.0	Production

Google Cloud, “MLOps: Continuous delivery and automation pipelines in machine learning”, “ML metadata and artifact tracking” / TensorFlow TFX Guide, “ML Metadata” (Apache-2.0)

Experiment Tracking

실험마다 설정·결과·산출물을 기록해야 비교하고 재현할 수 있다

- **Parameters**: 입력 설정 (learning rate, max_depth, 데이터 버전 …)
- **Metrics**: 결과 수치 (loss, accuracy, F1 …) — **step별로** 기록하면 학습 곡선이 됨
- **Artifacts**: 산출물 파일 (모델 가중치, 그래프, confusion matrix …)
- 한 번의 실행 = **Run**, 같은 목적의 Run 묶음 = **Experiment**
- 기록이 쌓이면: 실험 간 비교 · 최적 모델 선택 · 재현



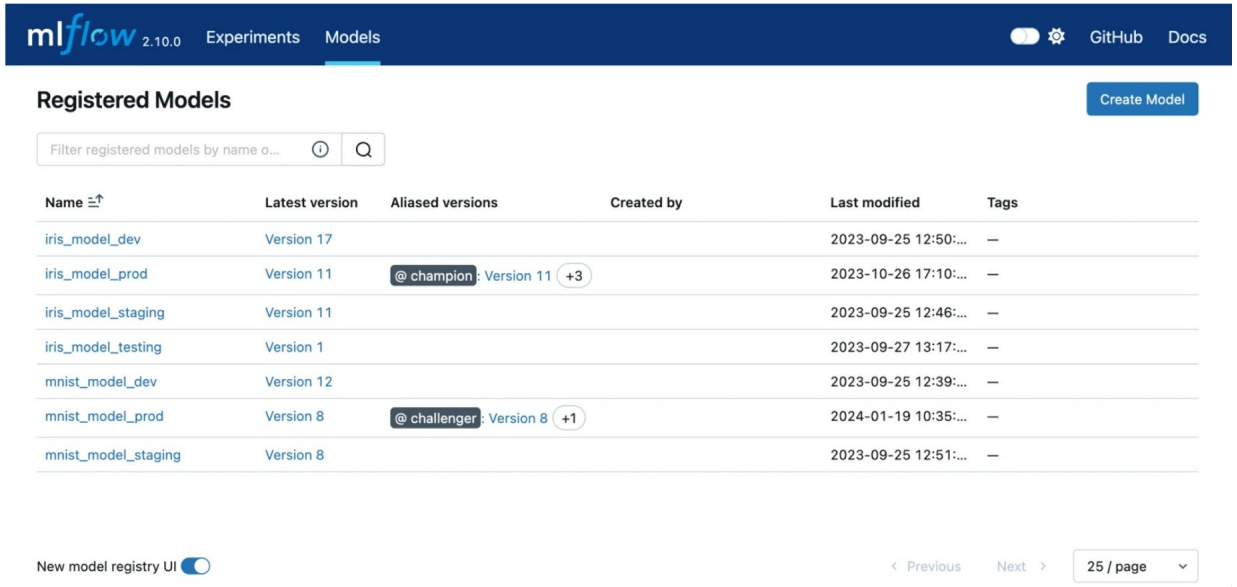
MLflow UI — 실험(Run)마다 기록된 val_loss를 step별 곡선으로 비교

MLflow Tracking Docs (mlflow.org/docs/latest/ml/tracking/), 이미지: github.com/mlflow/mlflow (Apache-2.0)

Model Registry

모델 버전과 배포 상태를 한 곳에서 관리해야 배포·롤백을 추적할 수 있다

- 모델을 등록하면 **버전 번호 자동 부여** (v1, v2, v3 …)
- 각 버전은 자신을 만든 **Run(코드·데이터·파라미터)**과 연결 → Lineage
- 배포 상태 관리
  - 과거 방식: Stage (None → Staging → Production → Archived)
  - MLflow 2.9+: **Alias** (예: @champion, @challenger) + Tag
- **Rollback**의 전제: 레지스트리에 이전 버전이 보존되어 있어야 복구 가능



MLflow Model Registry — @champion·@challenger alias로 서비스 중인 버전을 표시

MLflow Model Registry Docs (mlflow.org/docs/latest/ml/model-registry/workflow)

SECTION 03

AI 서비스를 위한 Database

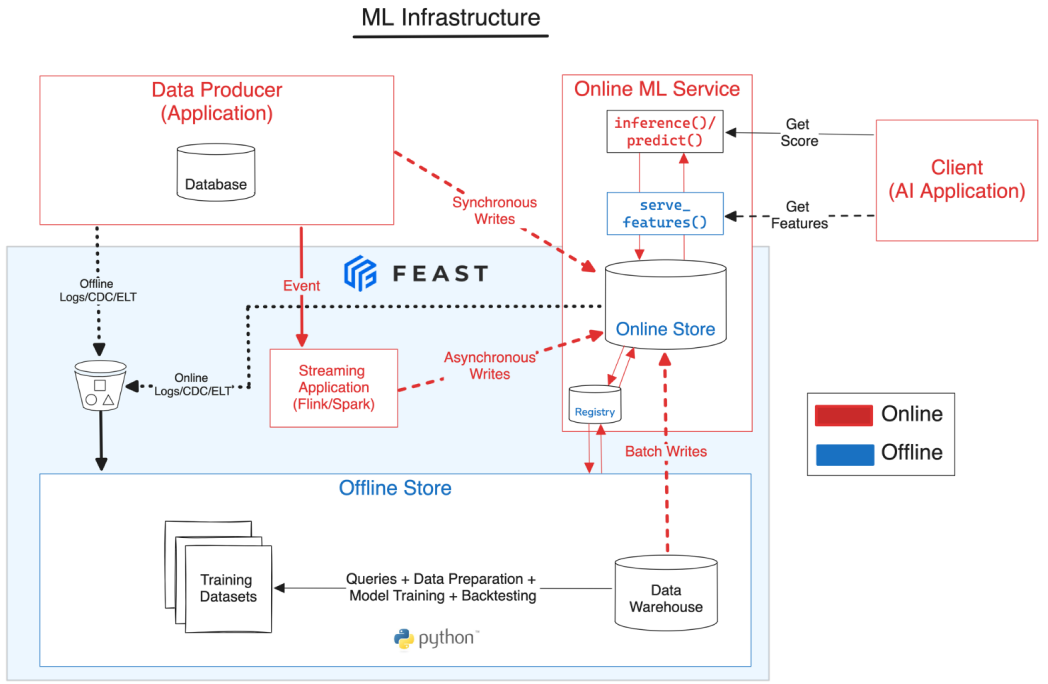
AI 서비스는 어떤 데이터를 어디에 저장하고 조회하는가?

학습의 데이터 vs 서비스의 데이터

학습은 대량 처리량, 서비스는 낮은 지연시간이 필요해 저장소를 나눈다

	Offline (학습 단계)	Online (서비스 단계)
양	대량 (수백만~수십억 행)	요청당 소량 (한 사용자, 한 상품)
접근	배치로 한 번에 스캔	요청마다 즉시 조회
요구	**처리량 (Throughput)**	**지연시간 (Latency)** — 수~수십 ms
시점	과거 데이터	**지금** 데이터
DB 유형 (참고)	OLAP(Online Analytical Processing) · 열 지향 (BigQuery, Parquet)	OLTP(Online Transaction Processing) · Key-Value (PostgreSQL, Redis)

- 서비스 단계의 추가 요구: **다수 사용자의 동시 접근, 데이터의 지속적 축적** (로그·피드백)
- → 하나의 저장소로 두 요구를 동시에 만족시키기 어려움

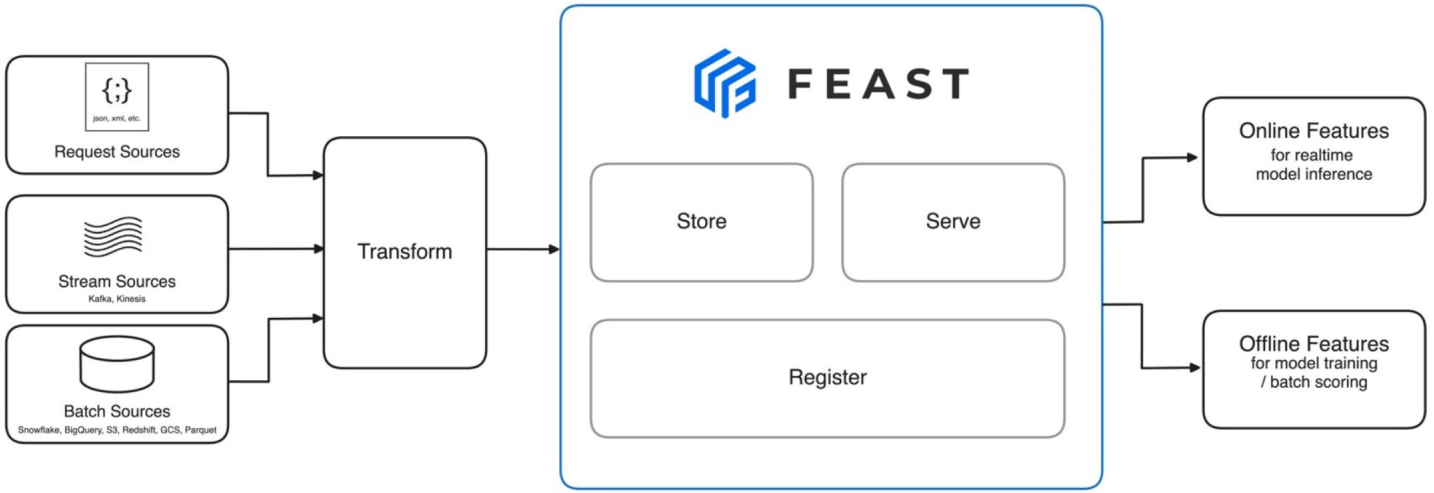


Feast: Online(빨강) 경로와 Offline(파랑) 경로

Feast Docs, “Model inference” architecture (github.com/feast-dev/feast, Apache-2.0) / Google Cloud MLOps 문서 “Feature store”

Feature Store — Training-Serving Skew 해결

학습·서빙 간 피처 계산 불일치를 해결하는 저장소



Feast — 하나의 피처 정의로 서빙용(Online)·학습용(Offline) 피처를 모두 제공

- 문제: 학습 때는 Python으로 "최근 7일 구매 횟수"를 계산, 서빙 때는 백엔드가 다른 코드로 계산 → **미묘하게 다른 값**
- Feature Store: 피처의 **정의 · 저장 · 조회를 한 곳에서 표준화**하는 중앙 저장소
  - **Offline Store**: 대량의 과거 피처 → 학습 데이터셋 생성 / **Online Store**: 엔티티별 **최신 값만** → 저지연 조회
  - 둘 다 **같은 피처 정의**에서 나옴 → 실험 · 재학습 · 서빙이 같은 소스 사용 → Skew 차단

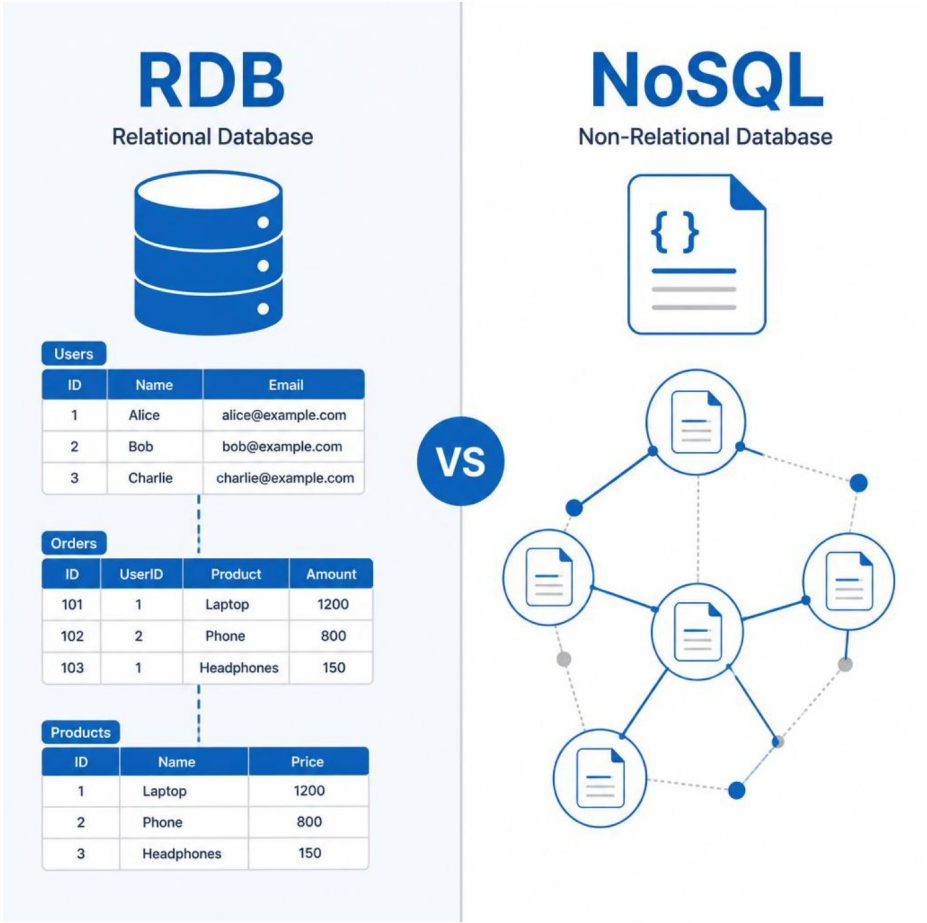
Feast (github.com/feast-dev/feast, Apache-2.0) / Google Cloud MLOps 문서 "Feature store"

기존 DB의 역할 — RDB와 NoSQL

정형 데이터와 트랜잭션 처리에는 여전히 RDB가 적합

	RDB (관계형)	NoSQL
구조	테이블 · 행 · 열, **고정 스키마**	Key-Value, Document, Wide-column, Graph 등 **유연한 스키마**
강점	**ACID 트랜잭션**, JOIN, 정합성	수평 확장, 대량 쓰기, 반정형 데이터
AI 서비스에서	사용자 정보, 주문·결제, 권한	이벤트 로그, 세션, 캐시, 피처 조회
예시	PostgreSQL, MySQL	Redis, MongoDB, Cassandra, DynamoDB

- ACID: 원자성 · 일관성 · 격리성 · 지속성 → 여러 테이블에 걸친 변경이 **전부 반영되거나 전부 취소**됨



RDB: 테이블 간 관계(키)로 연결 · NoSQL: 관계형 스키마 없이 저장 (개념도)

PostgreSQL Docs / MongoDB Docs, “Data Modeling” / Pinecone, “What is a Vector Database”

기존 DB로 처리하기 어려운 질의 — 벡터 유사도 검색

일부 AI 서비스는 “비슷한 것”을 찾는 질의를 요구하고, 기존 인덱스로는 이를 빠르게 처리할 수 없다

① 요구가 달라지는 경우

“같은 값” → “비슷한 것”

- 기존: WHERE user_id = 42 — 값이 일치하는 행 조회
- 일부 AI 서비스: “이 사진과 **가장 비슷한** 상품 10개” (의미 기반 검색, RAG 문서 검색, 유사 상품 추천)
- 데이터를 고차원 벡터로 바꾸고, 질의 벡터와 **거리가 가장 가까운 k개**를 찾는 질의 (k-NN, k-Nearest Neighbors)

→

② 기존 DB의 문제

기존 인덱스로는 빨리 찾을 수 없다

- B-tree는 값을 **1차원 순서**로 정렬해 O(log N) 탐색 → 고차원 벡터의 거리 순서에는 적용 불가
- 결국 모든 행과 거리 계산(**전수 비교**) → 데이터가 많아지면 지연시간 급증
- KD-tree 등 공간 분할 인덱스도 고차원에서는 전수 비교와 비슷해짐 (**차원의 저주**)

③ **그래서 필요한 것** — 이어지는 내용

Embedding 26쪽	**유사도 metric** 27쪽	**ANN 인덱스** 28–29쪽
데이터를 벡터로 어떻게 바꾸나	무엇을 “가깝다”고 볼 것인가	정확도를 약간 양보하고 빠르게 찾기

Beyer et al., “When Is Nearest Neighbor Meaningful?”, ICDT 1999

Embedding — CLIP 예시

텍스트·이미지 등을 같은 벡터 공간의 숫자 벡터로 변환

- Embedding: 텍스트·이미지·오디오를 **의미가 담긴 고정 길이 숫자 벡터**로 변환
- 의미가 비슷하면 벡터 공간에서 **가까이** 위치
- CLIP(Contrastive Language–Image Pre-training): 이미지 인코더와 텍스트 인코더가 **같은 공간**을 학습 → **Cross-modal Retrieval**이 벡터 검색 한 번으로 가능

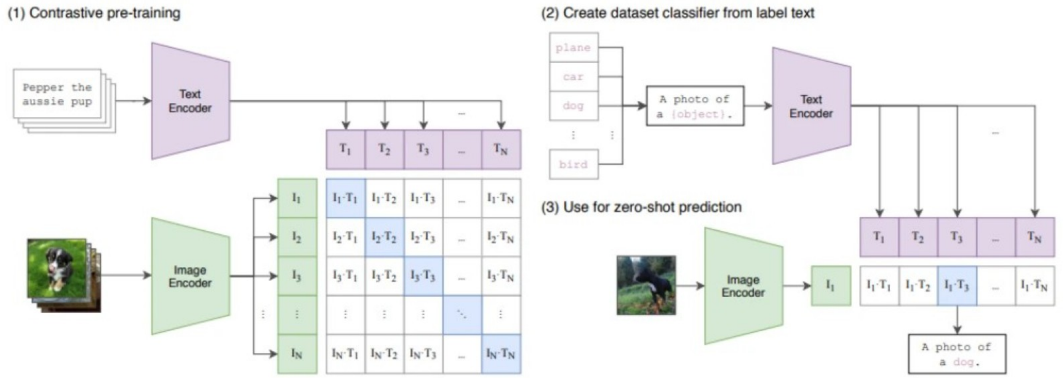


Figure 1. Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

CLIP Fig.1 — 짝이 맞는 이미지·텍스트 임베딩(대각선)은 가깝게, 나머지는 멀게 학습

주의: **"Vector라는 형식은 같아도, Vector Space의 의미가 같다는 보장은 없다"**

- 벡터 DB는 **거리만 계산**하며, 벡터의 의미는 **임베딩 모델**이 결정
- 임베딩 모델을 바꾸면 **기존 인덱스 전체를 다시 만들어야** 한다 → 임베딩 모델 버전 = 인덱스 버전

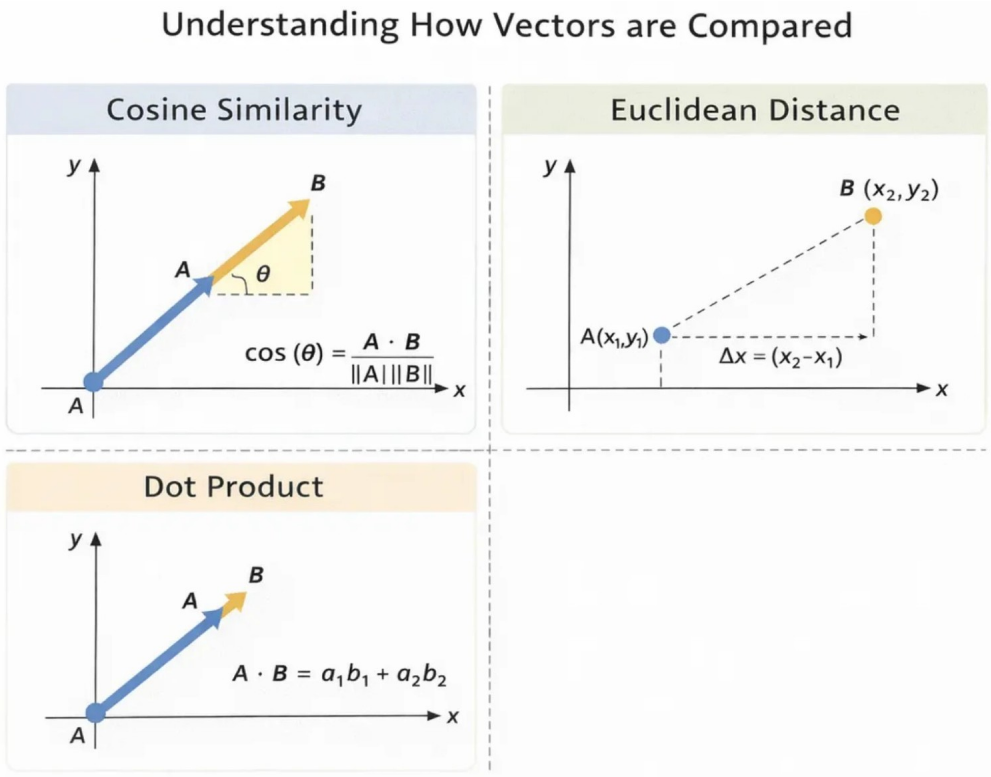
Radford et al., "Learning Transferable Visual Models From Natural Language Supervision", ICML 2021, Figure 1 / WEEK5 Basic Multimodal

유사도를 재는 세 가지 방법

| 임베딩 모델이 학습된 metric을 그대로 써야 한다

Metric	수식	의미	범위
Cosine Similarity	a·b / (‖ a ‖ ‖ b ‖)	**방향**이 얼마나 같은가	[-1, 1]
Euclidean (L2)	‖ a − b ‖ ₂	직선 **거리**	[0, ∞)
Dot Product	a·b	방향 × **크기**	(-∞, ∞)

- **L2 정규화**(길이 1)하면 ‖ a − b ‖ ² = 2 − 2cos(a, b) → **세 metric의 순위가 같아진다**
- 원칙: **임베딩 모델이 학습된 metric을 그대로** 사용 (CLIP은 정규화 후 cosine)



같은 두 벡터라도 cosine은 각도, L2는 직선거리, dot product는 각도와 크기를 함께 반영

Pinecone, “Vector Similarity Explained” / 이미지: medium.com/@mailpraveenreddy.c, “Understanding how vectors are compared”

ANN — 근사 최근접 이웃 검색

모든 벡터를 비교하지 않고, 후보만 골라 비교해 빠르게 찾는다

Exact k-NN (전수 비교)

모든 벡터와 거리를 계산

- 결과는 항상 정확
- 데이터가 많아질수록 느려짐

ANN (Approximate Nearest Neighbor)

미리 만든 인덱스로 후보 일부만 비교

- 진짜 top-k 중 일부를 놓칠 수 있음
- 대신 **훨씬 빠름**

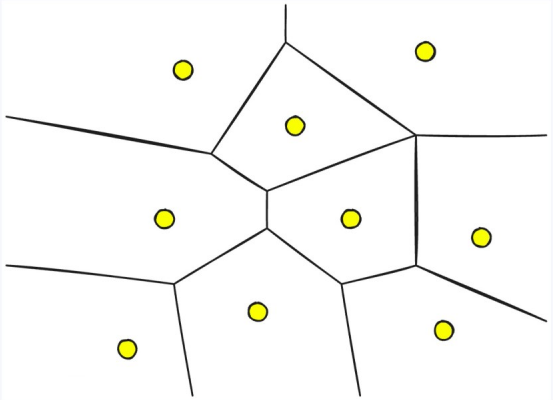
"가장 비슷한 상품 10개"는 **거의 맞으면 충분**한 경우가 많아, 대규모 벡터 검색은 ANN을 기본으로 사용

Aumüller, Bernhardsson, Faithfull, "ANN-Benchmarks", Information Systems 2020 / Pinecone, "What is a Vector Database"

ANN은 어떻게 빨라지나 — 대표적인 세 가지 방법

비교할 후보를 줄이거나(IVF·HNSW), 한 번 비교하는 비용을 줄인다(PQ)

① 공간을 나눈다



IVF (Inverted File Index)

비슷한 벡터끼리 구역으로 묶어 두고, 질의와 가까운 **구역 몇 개만** 탐색

② 그래프를 따라간다

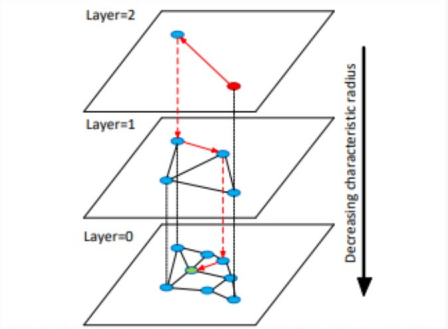
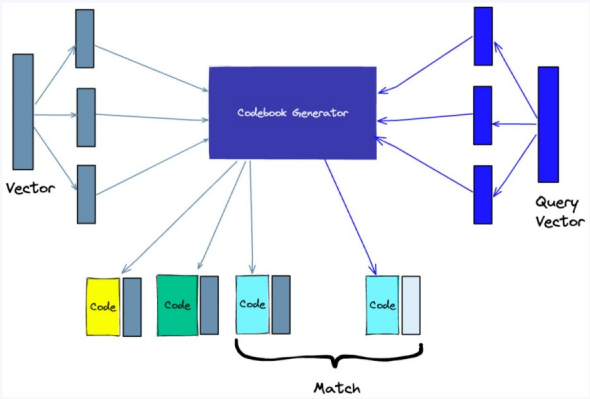


Fig. 1. Illustration of the Hierarchical NSW idea. The search starts from an element from the top layer (shown red). Red arrows show direction of the greedy algorithm from the entry point to the query (shown green).

HNSW (Hierarchical Navigable Small World)

가까운 벡터끼리 연결한 그래프에서, **더 가까운 이웃으로 이동**하며 탐색

③ 벡터를 압축한다



PQ (Product Quantization)

벡터를 조각내 짧은 코드로 바꿔 **메모리와 거리 계산 비용**을 줄임

①·②는 **비교할 후보 수**를, ③은 **한 번 비교하는 비용**을 줄임 → 실제로는 IVF + PQ처럼 조합해서 사용

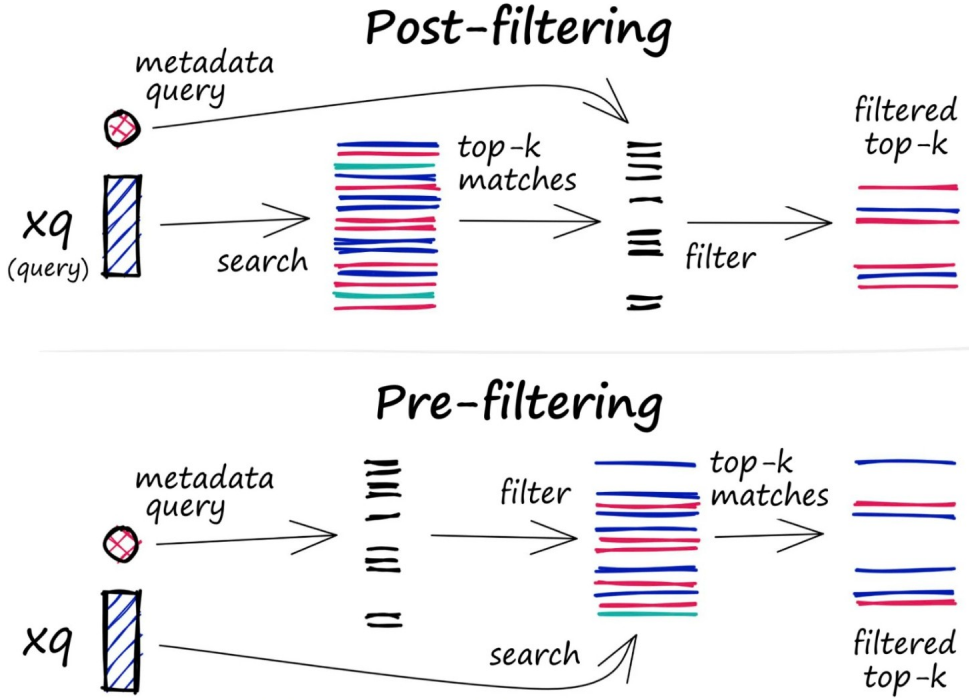
Pinecone, “What is a Vector Database” / Malkov & Yashunin, IEEE TPAMI 2020, Fig. 1 / Jégou et al., IEEE TPAMI 2011

Metadata Filtering

“비슷한 것” 중 “조건에 맞는 것”만 찾으려면 필터를 언제 적용할지 정해야 한다

예: “빨간 운동화와 **비슷한** 상품” 중 **재고 있음 · 10만원 미만**인 것만
→ 벡터마다 메타데이터(카테고리·가격·날짜·권한)를 함께 저장해 두고 조건으로 거름

	Pre-filtering	Post-filtering
순서	조건으로 거르기 → 벡터 검색	벡터 검색(top-k) → 조건으로 거르기
문제	남은 후보가 적으면 ANN 인덱스를 쓰기 어려움	조건이 까다로우면 결과가 k개보다 적게 남음



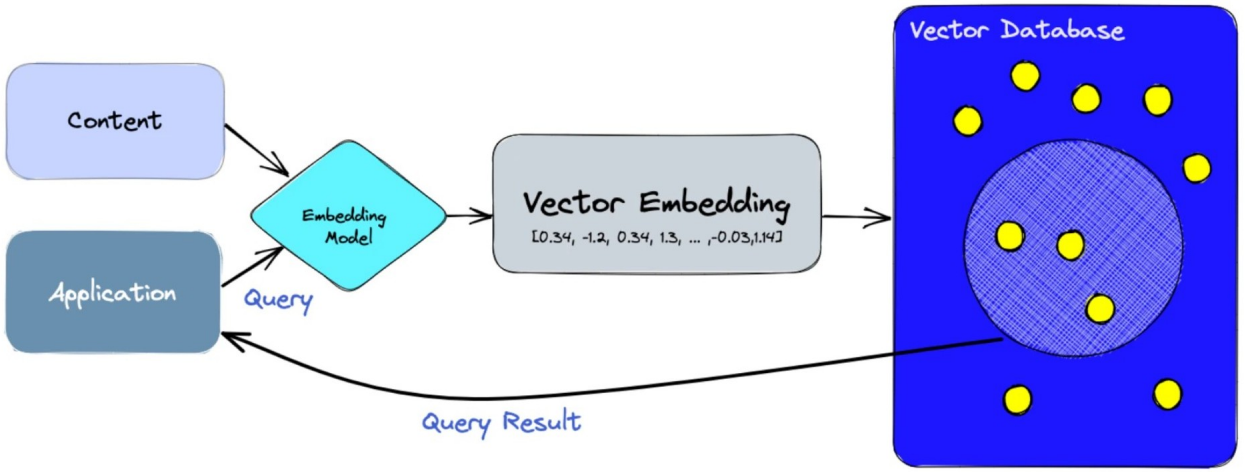
Post-filtering: 검색 후 거름 → 결과가 k개보다 적을 수 있음 · Pre-filtering: 거른 뒤 검색

RAG에서는 **권한 필터**(이 사용자가 볼 수 있는 문서만)가 필수

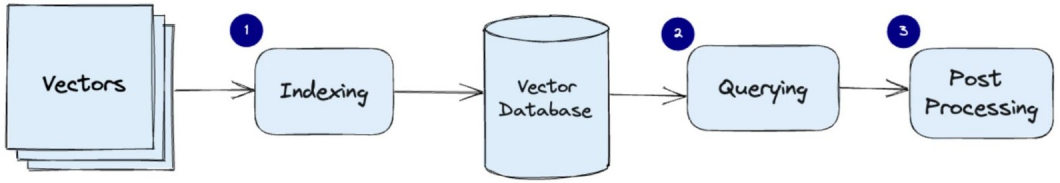
Pinecone, “What is a Vector Database” (Filtering)

Vector DB 활용 파이프라인

| 문서와 질의는 반드시 같은 임베딩 모델로 변환해야 한다



원천 데이터(Content)와 질의(Application)가 같은 임베딩 모델을 거쳐 Vector DB에서 만남



처리 순서: ① Indexing(적재) → ② Querying(검색) → ③ Post-processing(필터·재정렬)

- **Indexing (적재)**: 원천 데이터 수집 → **Chunking** → Embedding → Vector DB 저장 (+ 메타데이터, 원문 참조)
- **Query (검색)**: 사용자 질의 → **같은 임베딩 모델**로 Embedding → ANN 검색 (+ 필터) → Top-k → (선택) Re-ranking

Pinecone, “What is a Vector Database” / Gao et al., “Retrieval-Augmented Generation for Large Language Models: A Survey”, 2023

활용 ① RAG (Retrieval-Augmented Generation)

모델을 재학습하지 않고, 외부 문서 검색으로 최신 지식을 반영한다

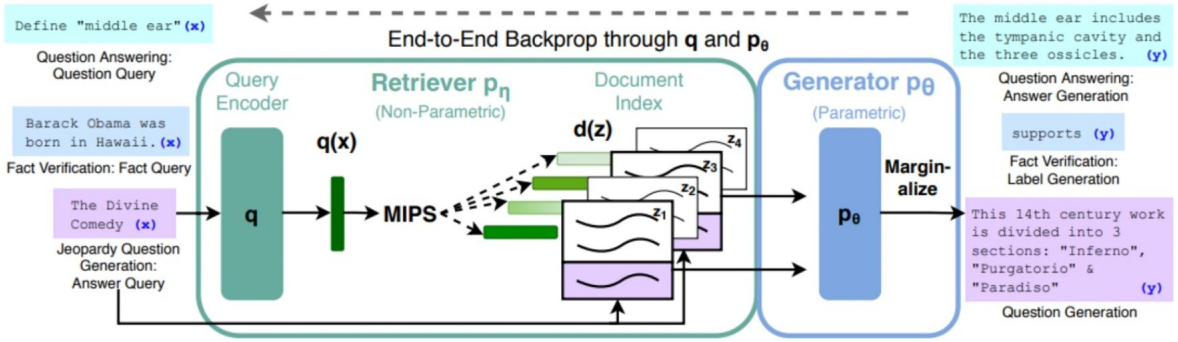


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder + Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x, we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i. For final prediction y, we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

RAG Fig.1 — Retriever가 질의와 가까운 문서 z를 찾고, Generator가 이를 참고해 답변을 생성

> *"We introduce RAG models where the parametric memory is a pre-trained seq2seq model and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever."*
>
> Parametric memory(사전학습 seq2seq 모델) + Non-parametric memory(Wikipedia의 dense vector index, retriever로 접근)
>
> — Lewis et al., 2020, Abstract

- MLOps 관점: 지식이 바뀌면 **모델 재학습 없이 인덱스만 갱신** · 검색된 출처 제시 가능

Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS 2020, Figure 1 · Abstract

활용 ② 추천시스템

후보 생성 = 사용자 벡터와 가까운 아이템 검색

- 사용자 임베딩과 아이템 임베딩을 **같은 공간**에 → 사용자 벡터와 가까운 아이템 = 후보
- **후보 생성(candidate generation) → 랭킹(ranking)** 2단계 구조
- Two-tower 모델 + ANN = 대규모 후보 생성의 표준 구조

> 서빙 시에는 ANN 조회로 수백 개의 후보 영상을 생성한다.
>
> — Covington et al., RecSys 2016, Figure 3 caption

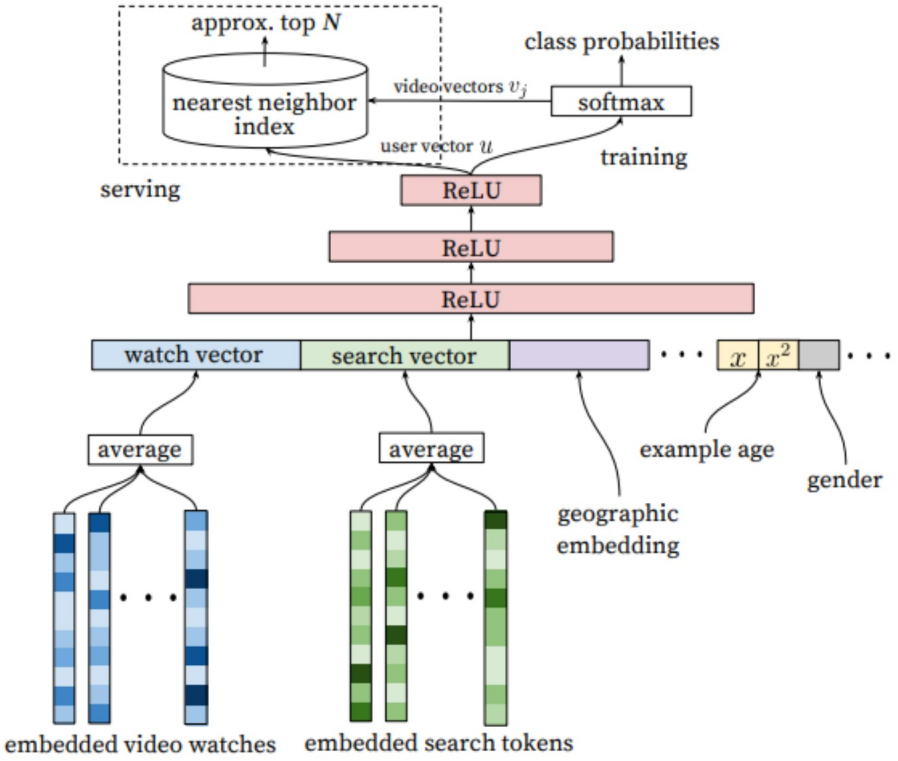


Figure 3: Deep candidate generation model architecture showing embedded sparse features concatenated with dense features. Embeddings are averaged before concatenation to transform variable sized bags of sparse IDs into fixed-width vectors suitable for input to the hidden layers. All hidden layers are fully connected. In training, a cross-entropy loss is minimized with gradient descent on the output of the sampled softmax. At serving, an approximate nearest neighbor lookup is performed to generate hundreds of candidate video recommendations.

학습(위 오른쪽)은 softmax로 다음 시청 영상을 예측하고, 서빙(위 왼쪽)은 사용자 벡터 u로 ANN 검색해 후보 N개를 뽑음

Covington, Adams, Sargin, “Deep Neural Networks for YouTube Recommendations”, RecSys 2016

Data Lake / Data Warehouse / Lakehouse

원본 보관은 Lake, 분석은 Warehouse, 둘을 합친 구조가 Lakehouse

	Data Lake	Data Warehouse	Lakehouse
정의	모든 원본 데이터를 형식 그대로 쌓아 두는 저장소 → **"raw 보관"**	분석용으로 정제한 테이블을 SQL로 빠르게 조회하는 DB → **"빠르고 정확한 분석"**	레이크 파일 위에 트랜잭션·버전 관리를 더한 구조 → **"보관과 분석을 한 곳에서"**
저장 형태	원본 그대로 (정형·반정형·비정형)	정제·구조화된 테이블	레이크 위에 테이블 계층
스키마	읽을 때 적용 (Schema-on-read)	쓸 때 적용 (Schema-on-write)	둘 다
강점	저비용·유연, 이미지·로그 저장 가능	빠른 분석 SQL, 신뢰성	유연성 + 트랜잭션·버전
예시	Amazon S3, Google Cloud Storage + Parquet	BigQuery, Snowflake, Redshift	Delta Lake, Apache Iceberg

- ML 관점: 레이크는 **원천 보관소** (전처리를 바꿔 재가공 가능 → 재현성), 웨어하우스는 **분석·피처 생성의 출발점**

Armbrust et al., "Lakehouse", CIDR 2021

Polyglot Persistence

| 용도별로 서로 다른 저장소를 조합하는 설계



Martin Fowler — 기능마다 다른 저장소
(Redis·RDBMS·Riak·Neo4j·MongoDB·Cassandra)를 쓰는 예

예시: AI 쇼핑몰 검색 요청 1회가 거치는 저장소

순서	처리	저장소
1	로그인 사용자 정보·권한 확인	PostgreSQL (RDB)
2	최근 행동 기반 온라인 피처 조회	Redis (Online Store)
3	질의 임베딩과 가까운 상품 검색 + 재고 필터	Vector DB
4	요청·클릭 로그 적재 (추후 학습용)	Data Lake (Amazon S3 등)

용도별 성능 최적화가 가능하지만, **저장소 간 동기화·운영 복잡도**가 증가

Martin Fowler, “PolyglotPersistence” (martinfowler.com/bliki/PolyglotPersistence.html) / pgvector (github.com/pgvector/pgvector)

SECTION 04

모델의 서비스화

학습된 모델을 시스템에서 호출할 수 있도록 하는 방법

Packaging

모델 파일만으로는 실행 환경이 재현되지 않음

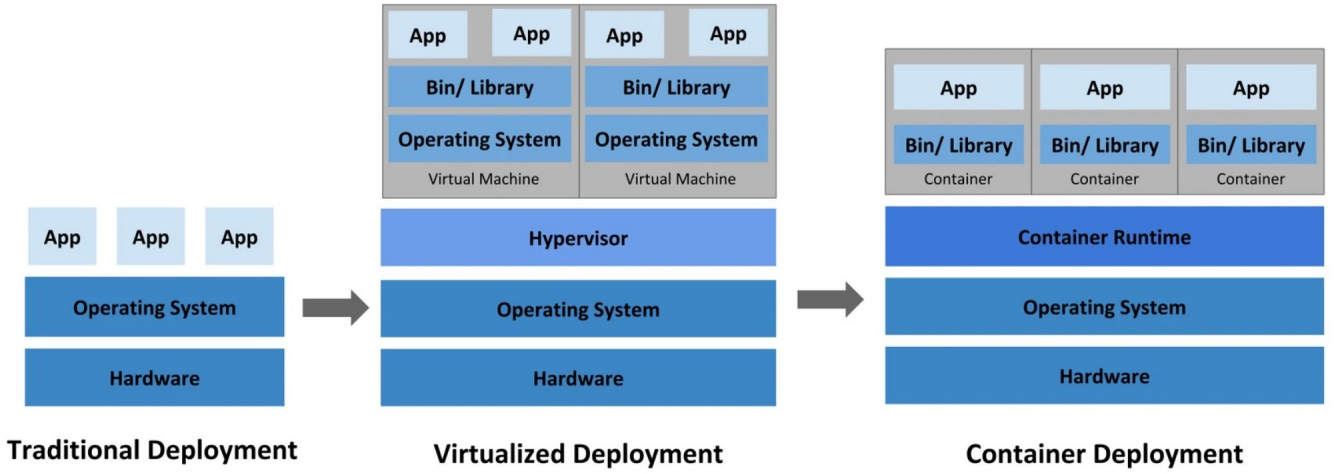
- 실행에 필요한 것: **모델 가중치 + 전처리·추론 코드 + 라이브러리 (정확한 버전) + 런타임 (Python, CUDA(NVIDIA GPU 연산 플랫폼), OS 라이브러리)**
- 하나라도 다르면: 에러, 혹은 **에러 없이 다른 결과**
- Packaging = 이 전부를 **하나의 실행 가능한 묶음**으로 만드는 것 → 목표: **개발 환경 = 테스트 환경 = 운영 환경**

구성요소	다르면 생기는 일 (예)
라이브러리 버전	scikit-learn 버전이 달라 pickle 모델 로드 실패 · 경고
전처리 코드	토크나이저 버전 차이로 토큰 분할이 달라짐
이미지 처리 라이브러리	PIL vs OpenCV 리사이즈 보간 방식 차이 → 입력 픽셀값이 달라짐
런타임 (CUDA 등)	GPU 드라이버·CUDA 불일치로 실행 불가

Google Cloud MLOps 문서 (Level 1: containerized components) / BentoML (github.com/bentoml/BentoML)

Image와 Container

| 컨테이너는 OS 커널을 공유해 VM보다 가볍고, 어디서나 같은 환경으로 실행된다



Traditional → Virtualized → Container Deployment

- **Virtualized (VM)**: VM마다 **자기 OS** → 격리는 좋지만 무거움 / **Container**: **호스트 OS 커널을 공유**, 앱 + 라이브러리만 격리 → 가볍고 어디서나 동일하게 실행
- **Image**: 실행에 필요한 모든 것을 담은 **읽기 전용 템플릿** (Dockerfile로 정의)
- **Container**: 이미지를 **실행한 인스턴스**. 하나의 이미지로 여러 컨테이너 실행 가능

Kubernetes Documentation, “Overview” (CC BY 4.0) / Docker Docs

Batch Prediction vs Online Prediction

| 예측에 요청 직전의 정보가 필요 없다면 Batch로 충분하다

	Batch Inference	Real-time (Online) Inference
실행 시점	정해진 시간에 대량으로 한꺼번에	요청이 올 때마다 즉시
예시	매일 새벽 전체 고객 이탈 확률 계산 → DB에 저장	사용자가 앱을 여는 순간 추천 생성
핵심 지표	**Throughput** (시간당 처리량)	**Latency** (응답 시간, 특히 p99: 가장 느린 1% 요청의 응답 시간)
피처	전날까지의 데이터	**요청 시점의 최신 데이터**
비용	실행 시에만 자원 사용 → 저렴	항상 서버 대기 → 비쌈
실패 시	재실행으로 복구	사용자에게 즉시 영향

- 판단 기준: **“예측이 요청 직전의 정보에 의존하는가?”** 그렇지 않으면 Batch로 충분

Google Cloud MLOps 문서 (Model serving) / BentoML Blog, “Unifying Real-Time and Batch Inference”

REST API로 모델 서빙

모델을 HTTP로 호출 가능한 서비스로 제공

- 원리: 모델을 **HTTP 요청으로 호출 가능한 API** 뒤에 배치
- 요청 (JSON) → 입력 **검증** → 전처리 → 추론 → 후처리 → 응답 (JSON)
- 모델 교체가 자유로움
- 도구 예시: **FastAPI** (타입 기반 입력 검증), **BentoML** (패키징·배칭·컨테이너화)

```
POST /predict
{"age": 27, "tenure_months": 14,
"monthly_spend": 52000}

200 OK
{"churn_prob": 0.83,
"model_version": "v12"}
```

응답에 모델 버전 포함 → 어떤 모델의 예측인지 추적 가능

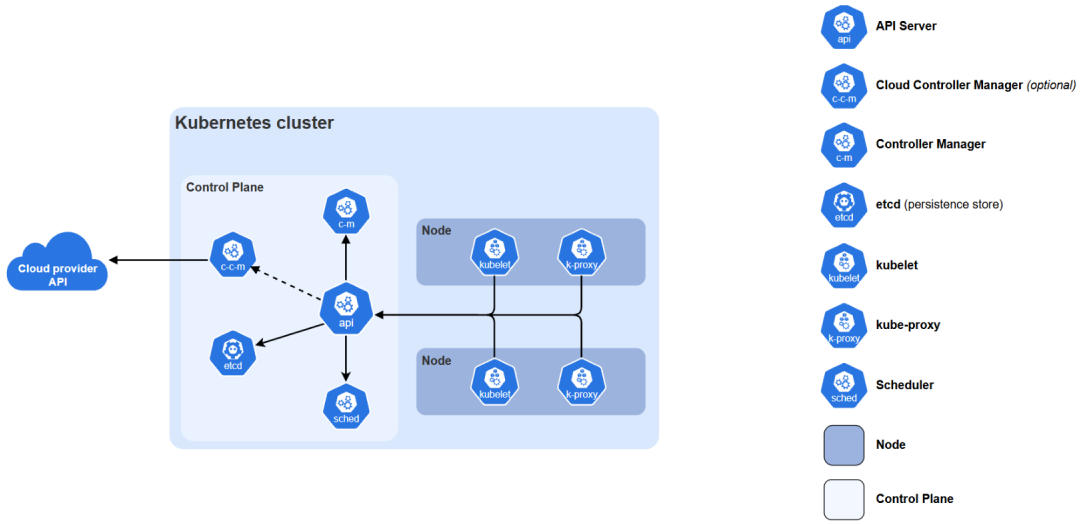
FastAPI Docs (fastapi.tiangolo.com) / BentoML (github.com/bentoml/BentoML)

배포와 확장 — Kubernetes의 원리

Desired State를 선언하면 Kubernetes가 현재 상태를 지속적으로 이에 맞춤

> Kubernetes는 현재 상태가 선언된 desired state에 도달하도록 지속적으로 조정하는, 독립적이고 조합 가능한(composable) control process의 집합이다.
>
> — Kubernetes Docs, “Overview — What Kubernetes is not”

- 배포 흐름: Registry에서 모델 선택 (@champion) → 서빙 **이미지 빌드** → 여러 노드에 **컨테이너 실행**
  - “이 이미지로 컨테이너 3개” 선언 → 장애 발생 시 자동 재시작 (**Self-healing**)
  - 부하가 높아지면 컨테이너 수 자동 증가 (**Horizontal Scaling**)
  - 새 버전으로 **점진 교체 (Rollout)**, 문제 시 **Rollback** · 준비되지 않은 컨테이너에는 트래픽 차단



Control Plane(API Server·Scheduler 등)이 각 Node의 컨테이너를 desired state에 맞게 관리

Kubernetes Documentation, “Overview”, “Kubernetes Components” (kubernetes.io, CC BY 4.0)

배포 전략 — 새 모델의 안전한 배포 방법

| 오프라인 검증 통과 후, 실제 트래픽에서 한 번 더 검증한다

전략	방식	사용자 영향	얻는 것
Shadow	새 모델에 트래픽을 **복제**해 보내되 응답은 사용하지 않음	없음	실제 입력에서의 지연·에러·예측 분포
Canary	트래픽의 **일부 (예: 5%)**만 새 모델로 → 점진 확대	일부	소규모 실사용 검증, 문제 시 빠른 철회
Blue-Green	구·신 환경을 둘 다 띄우고 **한 번에 전환**	전환 순간	즉시 롤백 가능 (트래픽 전환만 되돌림)
A/B Test	사용자를 무작위로 나눠 **비즈니스 지표를 통계적으로 비교**	실험군	성능 개선 여부에 대한 인과적 검증

- 공통 안전장치: **Rollback** — 레지스트리에 이전 버전(alias)이 남아 있어 포인터 변경만으로 복구
- Canary = 안전성 확인 (짧게, 작게) / A/B = 효과 측정 (충분한 표본·기간)

Google Cloud MLOps 문서 (online model validation) / Kubernetes Tutorial, “Deploy a Release Using a Canary Deployment”

SECTION 05

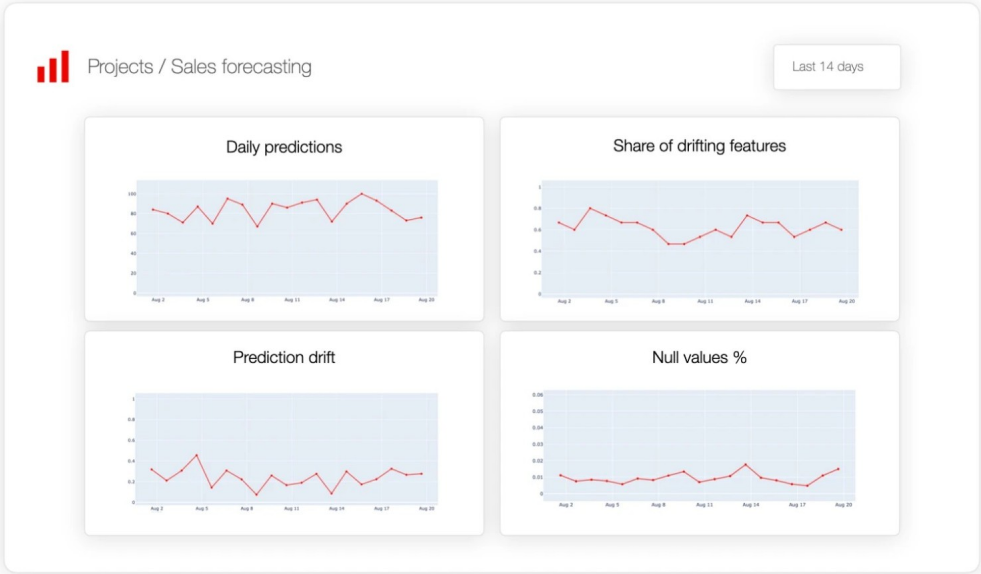
Monitoring

배포 이후 모델 성능은 어떻게 변하는가?

두 종류의 모니터링

서버가 정상이라고 해서 모델 예측이 맞는 것은 아니다

	시스템 모니터링	모델 모니터링
질문	서비스가 **정상 동작하는가?**	모델 예측이 **여전히 정확한가?**
지표	CPU·메모리·GPU, 응답 시간 (p99), 에러율, QPS(Queries Per Second)	입력 분포, 예측값 분포, (정답이 오면) 실제 성능
장애 모습	500 에러, 타임아웃 → **드러나는** 실패	200 OK로 틀린 답 → **드러나지 않는** 실패
도구 예시	Prometheus, Grafana	Evidently 등 ML 모니터링 도구

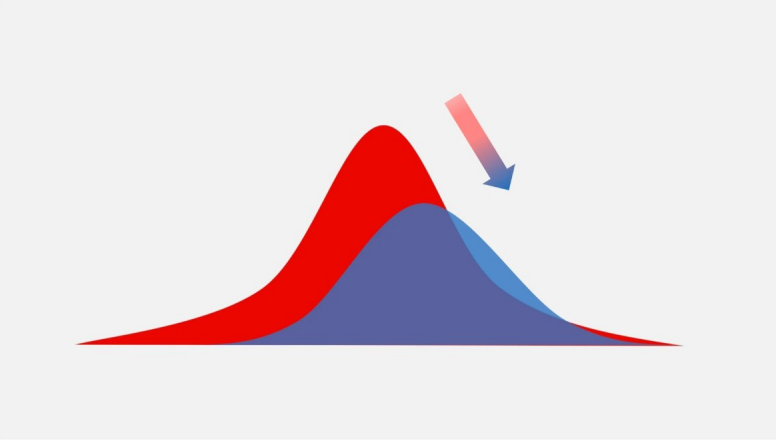


모델 모니터링 대시보드 예: 예측·drift·결측 비율

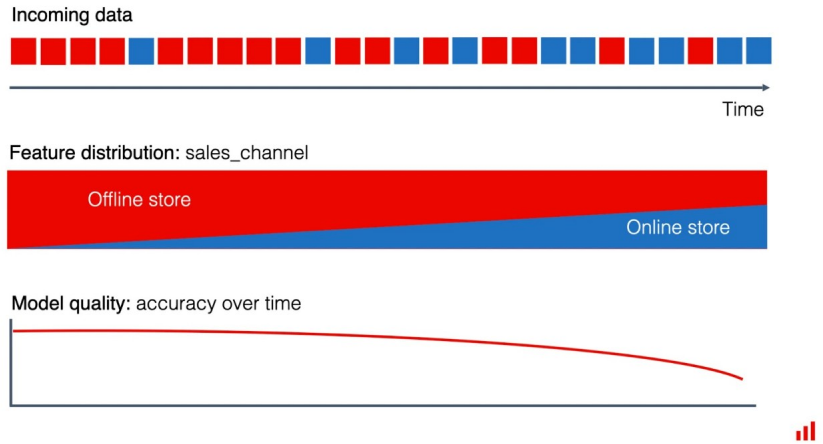
Evidently AI, “Model monitoring for ML in production” / Google Cloud MLOps 문서

Data Drift

입력 분포 P(X)가 학습 때와 달라지면, 학습 데이터에 적던 구간에서 성능이 떨어진다



학습 시점 분포(빨강) vs 운영 분포(파랑)



온라인 판매 비중 증가 → 정확도 서서히 하락

- 정의: 모델이 받는 **입력 피처 분포 P(X)**가 학습 때와 달라지는 것
- 예: 오프라인 매장 판매 위주로 학습한 수요예측 모델 → 앱 마케팅 후 **온라인 판매 비중 급증** → 학습 데이터에 적던 구간에서 성능 하락
- 입력–정답의 관계는 그대로일 수 있음. **모델이 학습한 분포에서 벗어난 영역의 데이터가 많아진 것**

Evidently AI, “What is data drift in ML, and how to detect and handle it”

Concept Drift

입력이 그대로여도 입력–정답 관계 P(Y|X)가 바뀌면 성능이 떨어진다

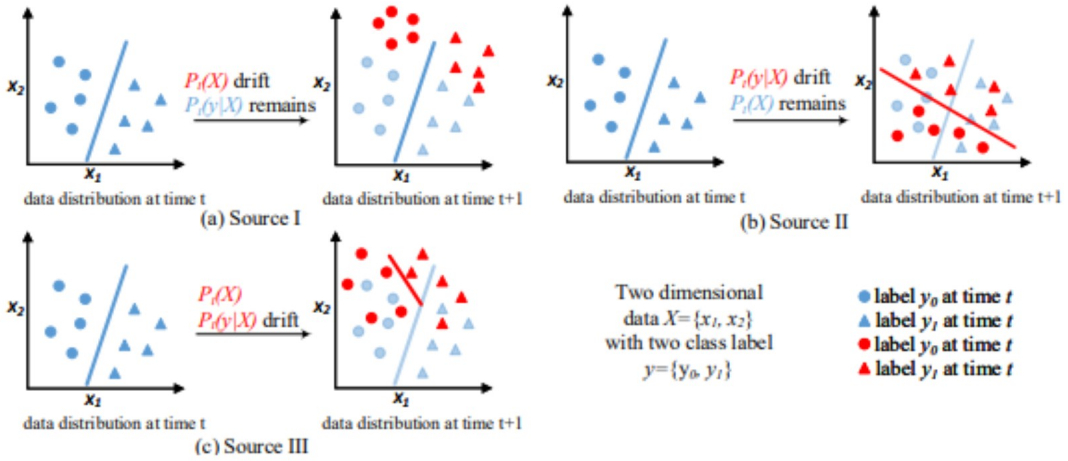


Fig. 3. Three sources of concept drift

Fig. 3 — 변화의 세 가지 원천 (Source I · II · III)

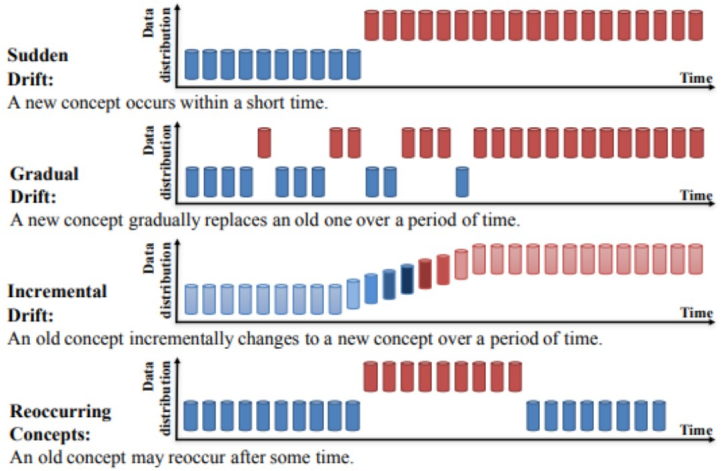


Fig. 4. An example of concept drift types.

Fig. 4 — 시간 패턴에 따른 유형: Sudden · Gradual · Incremental · Reoccurring

> *"Concept drift occurs at timestamp t + 1, if* $F_{0,t}(X, y) \neq F_{t+1,\infty}(X, y)$*, denoted as* $\exists t: P_t(X, y) \neq P_{t+1}(X, y)$*"*
>
> P(X, y) = P(X)·P(y|X) 이므로 변화의 원천은 세 가지: ① P(X)만 변화 (**virtual drift**) ② P(y|X)만 변화 (**actual drift**) ③ 둘 다 변화. 좁은 의미의 Concept Drift는 ②
>
> — Lu et al., "Learning under Concept Drift: A Review", IEEE TKDE 2019

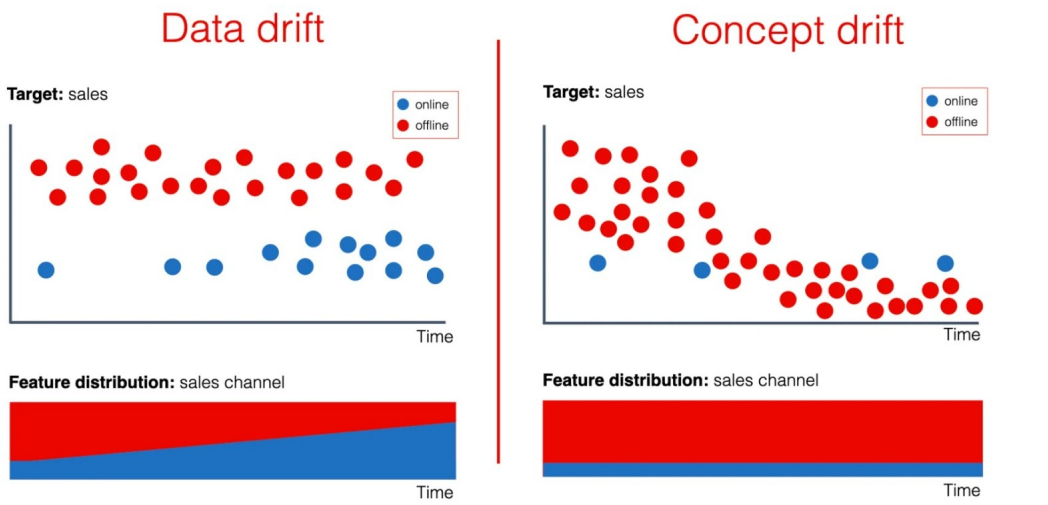
Lu et al., "Learning under Concept Drift: A Review", IEEE TKDE 2019 (arXiv:2004.05785), Fig. 3 · 4 / Gama et al., ACM CSUR 2014

Data Drift vs Concept Drift

Data Drift는 정답 없이 감지할 수 있지만, Concept Drift는 정답이 있어야 확실히 알 수 있다

	Data Drift	Concept Drift
무엇이 변하나	입력 분포 P(X)	입력–정답 관계 P(Y|X)
관련 용어	Covariate shift, Virtual drift	Real (Actual) concept drift
예	새로운 연령대 사용자 유입	경기 변화로 같은 신용 조건의 상환율이 달라짐
정답 없이 감지?	**가능** (입력만 보면 됨)	**어려움** (정답이 있어야 확실)
위험성	상대적으로 잘 보임	입력은 평소와 같지만 **높은 확신도의 오답**

- 둘은 동시에 일어나는 경우가 많고, Data Drift를 통해 Concept Drift를 의심할 수 있다



Data drift: 입력(판매 채널) 비중이 바뀜 · Concept drift: 입력 비중은 같은데 판매량(정답)이 달라짐

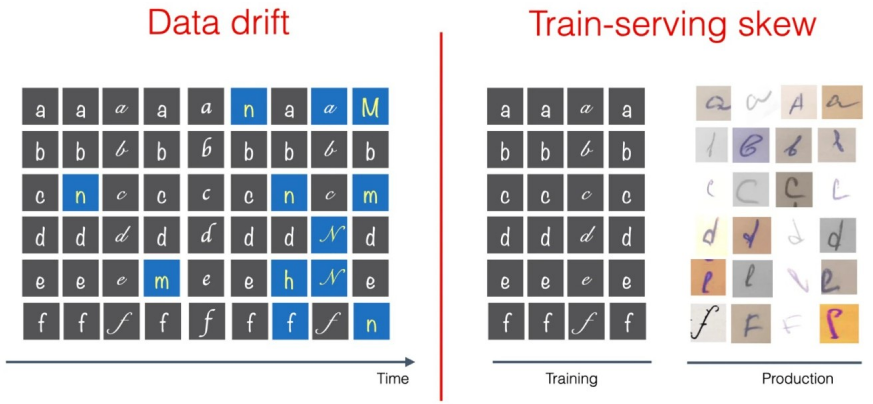
Evidently AI, “What is concept drift in ML” / Evidently AI, “What is data drift in ML, and how to detect and handle it”

Drift와 구분해야 할 개념

분포 변화처럼 보여도 파이프라인 차이·데이터 오류·개별 이상치일 수 있다

개념	핵심	Drift와의 차이
Training-Serving Skew	학습 데이터 vs 운영 데이터의 **불일치**	배포 **직후**부터 존재, 파이프라인 차이 등 환경 변화와 무관한 원인 포함
Data Quality 이슈	결측·오류·스키마 깨짐 (파이프라인 버그)	데이터가 **틀린 것** vs 정상 데이터의 **분포 변화**
Outlier	**개별** 샘플 수준의 이상	Drift는 **전체 분포** 수준의 변화

순서: **데이터 품질 검사 먼저 → 그 다음 Drift 검사** (품질 버그가 Drift처럼 보이는 경우가 흔함)

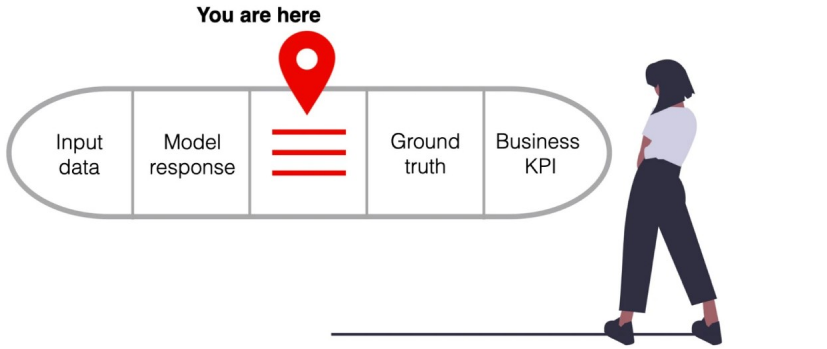


좌: 시간에 따른 점진적 변화(drift) · 우: 학습과 운영 데이터의 불일치(skew)

Evidently AI, “What is data drift in ML” (Data drift vs. Training-serving skew / Data quality / Outlier)

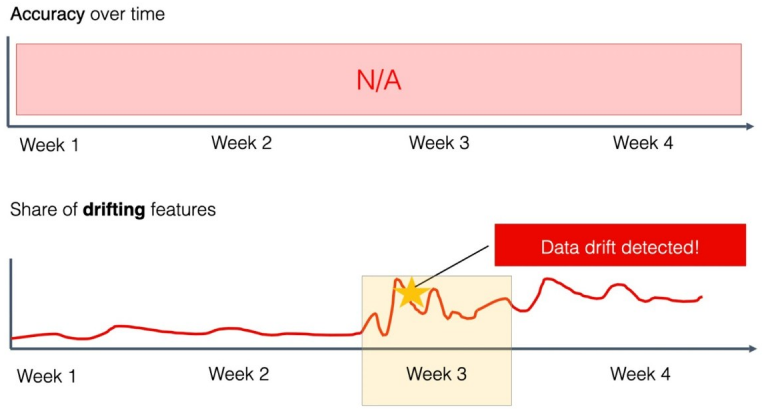
Label Delay (Feedback Delay) — 정답의 지연 도착

정답이 늦게 오므로, 그 전까지는 대리 지표로 성능 저하를 추정한다



- 성능을 재려면 **정답 (Label)**이 필요한데, 정답은 **지연되어** 온다
  - 대출 상환 여부: 수개월~수년 뒤
  - 사기 거래 여부: 이의 제기 후 몇 주 뒤
  - 추천 클릭: 수 초~수일
- 그 전까지는 성능이 저하된 모델이 계속 사용됨
- 대응: **정답 없이 볼 수 있는 대리 지표 (Proxy)**로 조기 경보
  - 입력 분포 (Data drift), 예측값 분포 (Prediction drift), 예측 확신도, 비즈니스 지표

Ground truth는 모델 응답 이후 지연되어 도착



정확도를 모를 때 drift 비율을 대리 지표로 사용

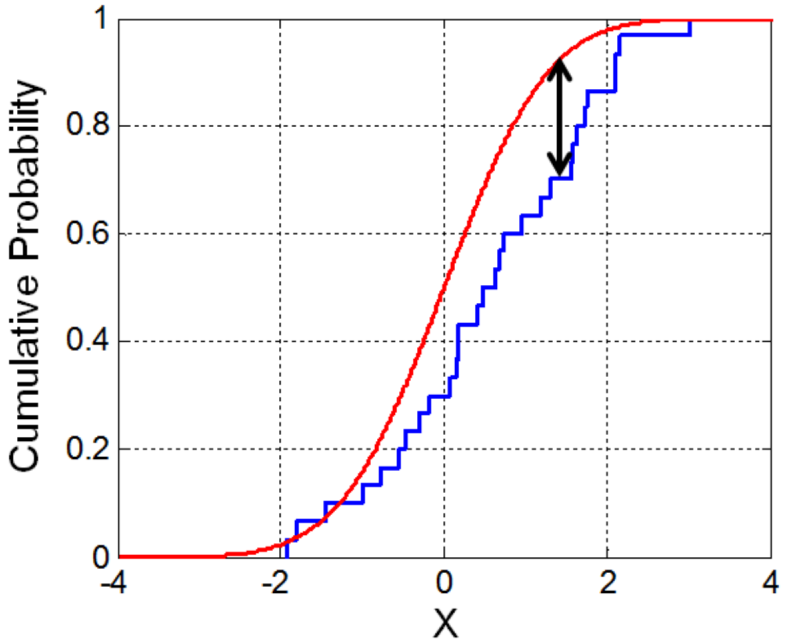
Evidently AI, “What is data drift in ML, and how to detect and handle it” (Feedback delay)

Drift는 어떻게 탐지하나

통계적 검정·거리 지표로 기준 분포와 현재 분포 비교

방법	대상	출력	특징
요약 통계	평균·분위수·결측률	값	단순, 도메인 지식 기반 임계값
KS(Kolmogorov–Smirnov) test	수치형	p-value	D = sup |F_ref(x) − F_cur(x)|
χ^2 test	범주형	p-value	범주 빈도 차이
PSI(Population Stability Index)	구간화한 분포	거리	업계 관행: <0.1 안정, 0.1–0.25 주의, >0.25 큰 변화
Wasserstein / JS(Jensen–Shannon)	수치형	거리	"얼마나" 변했는지 크기로
규칙 기반	비즈니스 규칙	경보	새 범주값 등장 등

- 표본이 크면 가설검정은 **아주 작은 차이도 유의** → 대용량에서는 거리 지표 선호
- Drift가 곧 성능 저하는 아님. **중요 피처의 Drift**에 우선순위



KS 통계량 D: 두 누적분포(CDF) 사이의 최대 수직 거리(화살표)

Evidently AI, "What is data drift in ML, and how to detect and handle it" / Wikimedia Commons, "KS_Example.png"

SECTION 06

자동화와 Continuous Training

재학습·검증·재배포 파이프라인은 어떻게 자동화하는가?

파이프라인 자동화의 필요성

작업 순서·실행 시점·재시도·기록을 사람이 아닌 오케스트레이터가 관리한다

- 데이터 수집 → 검증 → 전처리 → 학습 → 평가 → 등록 → 배포, 매번 **사람이 순서대로** 실행
- **Pipeline Orchestration**: 작업 간 **의존관계**를 정의하고, **정해진 시점/조건**에, **순서대로** 실행 · 실패 시 **재시도** · 실행 **기록**
- Pipeline Jungle에 대한 해법: 스크립트 뭉치 → **명시적인 그래프**

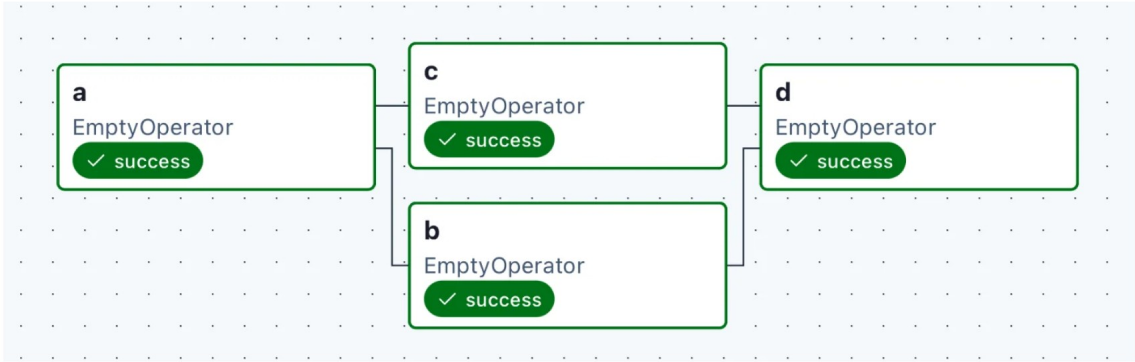
	수동 스크립트 실행	Orchestration
실행 순서	작업자의 기억에 의존	의존관계 그래프로 보장
실행 시점	사람이 직접 실행	스케줄 · 데이터 도착 · 트리거
실패 시	처음부터 재실행	실패한 Task만 재시도
기록	없음 또는 수기	실행 이력 · 로그 자동 저장

Google Cloud MLOps 문서 (Level 1: orchestrated experiment) / Sculley et al., 2015 (Pipeline Jungles)

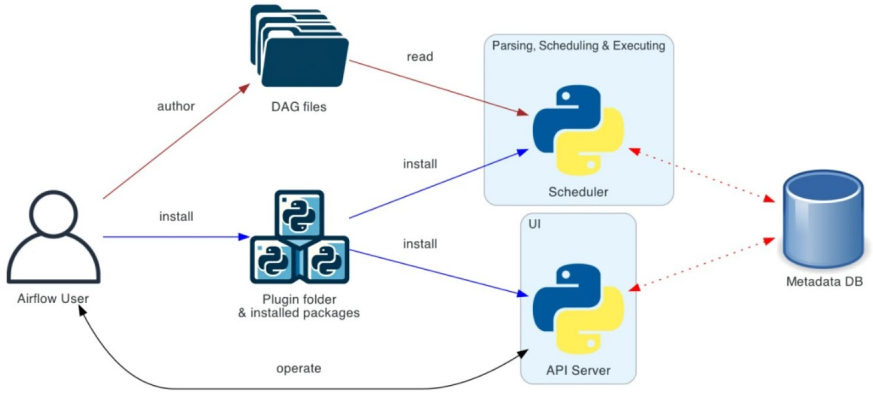
Airflow — DAG · Task · Scheduler

작업 간 의존관계를 DAG로 정의하면 Scheduler가 순서와 시점에 맞춰 실행한다

- **DAG** (Directed Acyclic Graph): 작업들과 그 **의존관계**. **순환이 없어야** 실행 순서(위상 정렬)가 정해진다
- **Task**: DAG 안의 **개별 작업** (예: extract, validate, train)
- **Scheduler**: DAG를 **언제** 실행할지 판단하고, 의존성이 충족된 Task를 실행 대기열에 올림
- 스케줄: 시간 기반 (cron) / **데이터 기반** (새 데이터 도착 시)
- 핵심 가치: 순서 보장 · 실패 재시도 · 실행 이력 · backfill



Airflow UI Graph view: a → (b, c) → d 의존관계



Airflow 기본 아키텍처: DAG files · Scheduler · API Server · Metadata DB

Apache Airflow Docs, Core Concepts — DAGs (airflow.apache.org) / 이미지: github.com/apache/airflow (Apache-2.0)

Continuous Training — 재학습 트리거

재학습 시점은 주기·새 데이터·성능 저하·분포 변화 중 서비스에 맞는 조건으로 정한다

트리거	설명	정답 필요?
① On demand	필요할 때 수동 실행	—
② On a schedule	매일 / 매주 / 매월	—
③ 새 학습 데이터 도착	새 데이터가 수집되면	—
④ 모델 성능 저하	성능이 기준 이하로 떨어지면	**필요**
⑤ 데이터 분포의 유의미한 변화	Drift 감지 시	불필요

- 트레이드오프: 재학습 주기가 짧을수록 최신 데이터를 반영하지만 **비용↑, 검증 부담↑, 불안정성↑**
- Drift 감지가 곧 재학습 조건은 아님: 데이터 품질 문제일 수 있으므로 **검증 단계를 먼저 수행**

Google Cloud MLOps 문서, “ML pipeline triggers” / Evidently AI, “What is data drift in ML, and how to detect and handle it”

CT 파이프라인

재학습된 모델의 성능 향상은 보장되지 않음

단계	내용	실패 시
① Data Validation	스키마 · 품질 · 분포 검사	중단 + 알림
② Retrain	새 데이터로 학습	—
③ Evaluate	테스트셋 지표 산출	—
④ Model Validation	Challenger vs Champion, 세그먼트별 성능, 서빙 호환성	Challenger 폐기
⑤ Registry 등록	@challenger alias 부여	—
⑥ 승인	사람 또는 규칙	보류
⑦ 배포	Shadow / Canary로 점진 투입	Rollback
⑧ Monitoring	시스템 지표 + Drift	Rollback (@champion 복귀)

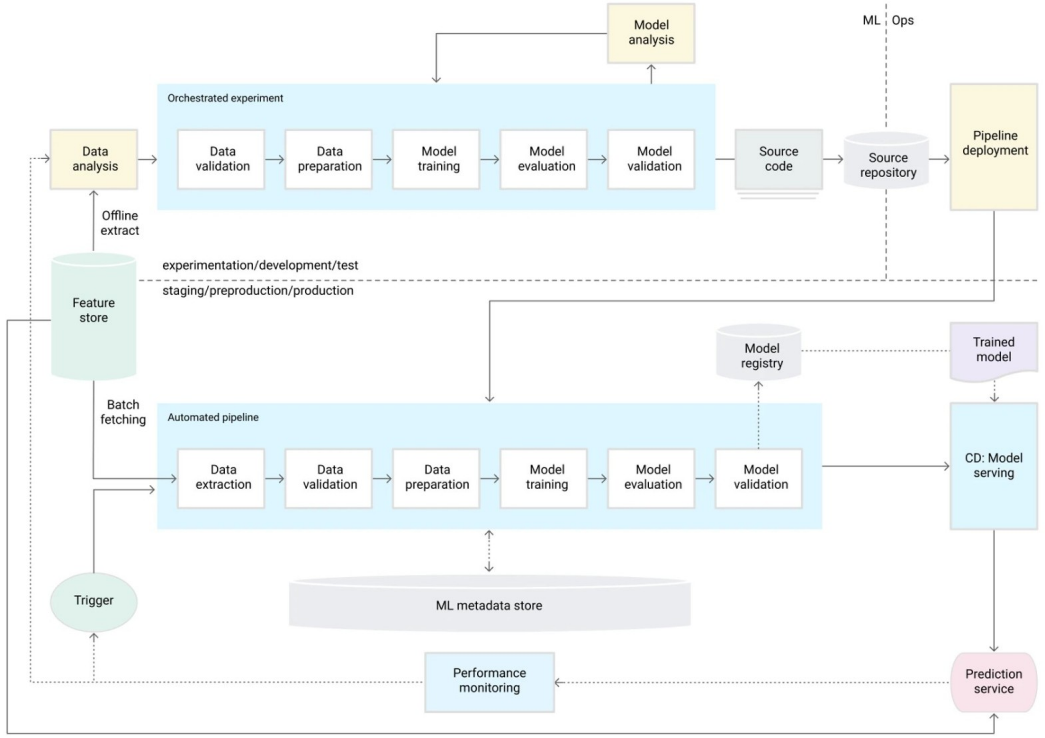


Figure 3 — 자동 파이프라인 안의 Data validation(①)과 Model validation(④) 단계

검증 단계 두 개: **Data Validation**(학습 전, 품질 미달 데이터 차단) + **Model Validation**(배포 전, 성능 미달 모델 차단)

Google Cloud MLOps 문서 (“Data and model validation”, Figure 3) / MLflow Model Registry (alias)

심화 — 전체 재학습의 대안

대형 모델은 전체 재학습 대신 일부 파라미터만 학습해 비용과 망각을 줄인다

- 전체 재학습의 두 문제: **비용** (대형 모델) + **Catastrophic Forgetting** (새 데이터로만 추가 학습하면 이전 성능 상실, McCloskey & Cohen 1989)

접근	아이디어	대표 연구
정규화 기반	**이전 태스크에 중요한 가중치의 변화에 페널티**	EWC (Kirkpatrick et al., 2017)
리플레이 기반	**예전 데이터 일부를 새 데이터와 섞어 학습**	Experience replay 계열
PEFT Parameter-Efficient Fine-Tuning	**원본은 고정하고 작은 모듈만 학습**	Adapter (Houlsby, 2019) LoRA (Hu, 2022)

EWC: Elastic Weight Consolidation · LoRA: Low-Rank Adaptation

> 구분: MLOps의 **CT**(같은 태스크를 새 데이터로 재학습)와 **Continual Learning**(여러 태스크를 순차 학습하며 잊지 않기)은 같은 개념이 아님

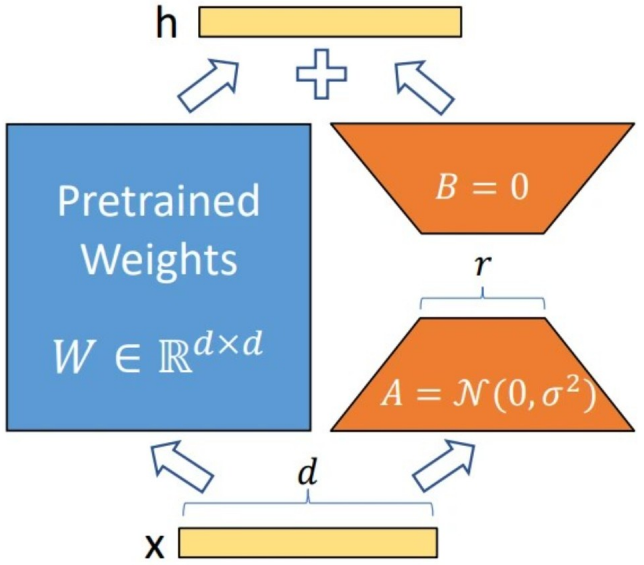


Figure 1: Our reparametrization. We only train A and B.

LoRA Figure 1: W 고정, A·B만 학습

Kirkpatrick et al., PNAS 2017 / Houlsby et al., ICML 2019 / Hu et al., “LoRA”, ICLR 2022 / Wang et al., TPAMI 2024

전체 파이프라인

데이터 → 학습 → 배포 → 모니터링 → 재학습이 하나의 순환 구조로 연결된다

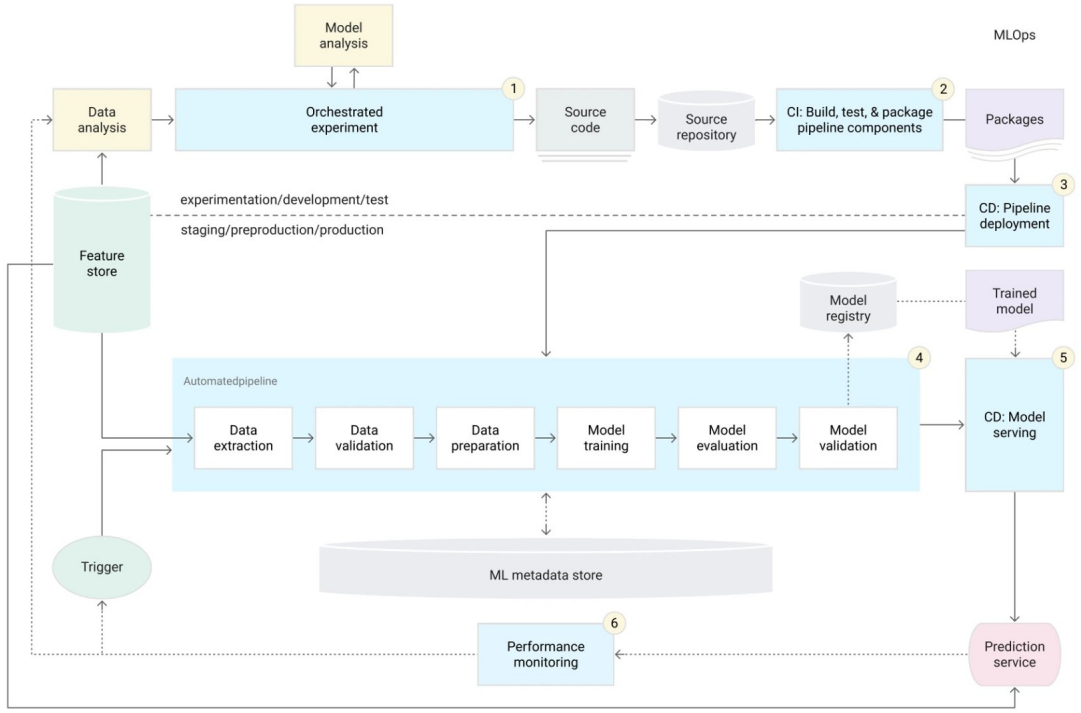


Figure 4 — 표의 각 단계가 그림의 구성 요소에 대응 (데이터 → 학습 → 배포 → 모니터링 → 재학습)

단계	개념 (파트)	도구 예시
원천 데이터	Data Lake / Warehouse (3)	S3, BigQuery
피처 준비	Feature Store Offline (3)	Feast
학습 파이프라인	Orchestration, Data Validation (6)	Airflow
실험 기록	Experiment Tracking (2)	MLflow Tracking
모델 등록	Model Registry, Validation (2·6)	MLflow Registry
패키징	Image / Container (4)	Docker
서빙	Online Serving, Online Store, Vector DB (3·4)	FastAPI, BentoML, K8s(Kubernetes)
점진 배포	Shadow / Canary (4)	—
모니터링	Monitoring, Drift (5)	Evidently
재학습 · 복구	CT Trigger, Rollback (6)	Airflow, Registry alias

Google Cloud, “MLOps: Continuous delivery and automation pipelines in machine learning”, Figure 4 (CC BY 4.0) / 각 도구 공식 문서

ML 시스템 구성 요소 다시 보기

Sculley et al. Figure 1의 구성 요소와 MLOps 개념의 대응

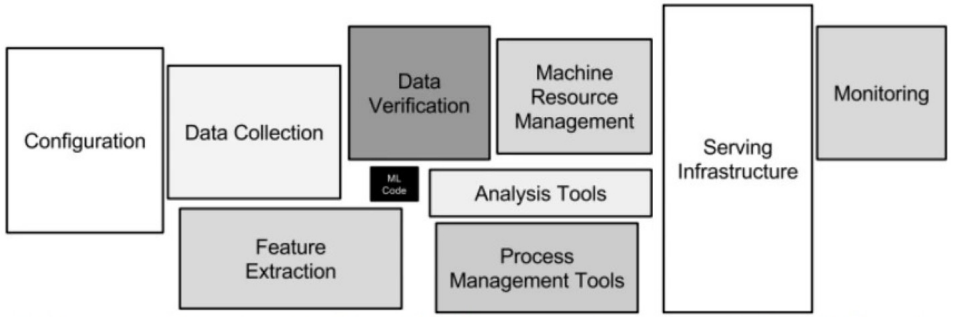


Figure 1: Only a small fraction of real-world ML systems is composed of the ML code, as shown by the small black box in the middle. The required surrounding infrastructure is vast and complex.

6쪽에서 본 Sculley et al. Fig.1 — 각 구성 요소를 이 발표에서 다룬 개념으로 대응

- Configuration → Registry · Versioning
- Data Collection · Verification → Data Validation · Feature Store
- Serving Infrastructure → Container · API · Kubernetes
- Monitoring → Drift · Label Delay
- Process Management → Airflow · CT

ML System = Code + Data + Model + Environment. **MLOps는 이 넷의 변화를 기록 · 통제 · 감지 · 대응한다.**

Sculley et al., NeurIPS 2015, Figure 1

Q&A

Quiz

(객관식) 다음을 읽고 올바른 선택지를 모두 골라주세요.

1. Online Prediction은 요청마다 즉시 예측하므로, 모든 서비스에서 Batch Prediction보다 항상 더 적합하다.
2. 벡터를 L2 정규화하면 코사인 유사도 기준 최근접 이웃과 유클리드 거리 기준 최근접 이웃의 순위가 같아진다.
3. 임베딩 모델을 더 좋은 모델로 교체하면, 기존 벡터 DB 인덱스는 그대로 두고 새 질의만 새 모델로 임베딩하면 된다.
4. Concept Drift는 입력 분포 P(X)가 그대로여도 발생할 수 있으므로, 입력 분포 모니터링만으로는 탐지하지 못할 수 있다.
5. Feature Store의 Online Store는 학습 데이터셋을 만들기 위해 대량의 과거 피처를 저장하는 곳이다.
6. 재학습된 모델은 최신 데이터를 반영하므로 별도 검증 없이 기존 모델을 대체해도 된다.

(서술형) DevOps의 CI/CD만으로는 ML 시스템을 운영하기에 부족한 이유를 설명하고, CT(Continuous Training)가 추가로 필요한 이유를 “코드가 바뀌지 않아도 모델이 낡는다”는 관점에서 서술하세요.

Reference

추천 Reference

- Google Cloud, MLOps: Continuous delivery and automation pipelines in ML — docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning
- 모두의 MLOps (한국어) — mlops-for-all.github.io/docs/welcome/
- Pinecone Learn, What is a Vector Database — pinecone.io/learn/vector-database/
- Evidently AI, What is data drift in ML — evidentlyai.com/ml-in-production/data-drift
- Stanford CS 329S: Machine Learning Systems Design — stanford-cs329s.github.io

추천 Paper

- Kreuzberger et al., Machine Learning Operations (MLOps): Overview, Definition, and Architecture — arxiv.org/abs/2205.02302
- Breck et al., The ML Test Score: A Rubric for ML Production Readiness — IEEE BigData 2017
- Pan, Wang, Li, Survey of Vector Database Management Systems — arxiv.org/abs/2310.14021
- Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks — arxiv.org/abs/2005.11401
- Lu et al., Learning under Concept Drift: A Review — arxiv.org/abs/2004.05785